



Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia Informática

Licenciatura em Engenharia Informática

Unidade Curricular de Laboratórios de Informática IV

Ano Letivo de 2025/2026

Sistema de Gestão do Clube Desportivo

Rui Miguel Gerales Branco da Cruz
a104355

Rafael Ferreira Rodrigues
a96640

David Filipe R
a104360

Fevereiro, 2026

Data da Receção	
Responsável	
Avaliação	
Observações	

Sistema de Gestão do Clube Desportivo

Rui Miguel Geral-
des Branco da Cruz
a104355

Rafael Ferreira Ro-
drigues
a96640

David Filipe Rocha
Figueiredo
a104360

Hugo Ferreira Soa-
res
a107293

Fevereiro, 2026

Resumo

<<O resumo tem como objectivo descrever de forma sucinta o trabalho realizado. Deverá conter uma pequena introdução, seguida por uma breve descrição do trabalho realizado e terminando com uma indicação sumária do seu estado final. Não deverá exceder as 400 palavras.>>

Área de Aplicação: Desenho e arquitetura de Sistemas de Software de Gestão.

Palavras-Chave: Java, Gestão de bases de dados, Arquitetura de Software, Engenharia de Requisitos, Conceção de Sistemas de Software, Vue.

Índice

1. Conceção e Engenharia de Requisitos Assistida por LLM	1
1.1. Análise de contexto e restrições	1
1.2. Definição do domínio do sistema	4
1.3. Identificação de stakeholders	6
1.4. Geração de user stories	7
1.5. Simulação de entrevistas com stakeholders	9
1.5.1. Entrevista 1: Sr. Joaquim “Quim” Barrela	9
1.5.2. Entrevista 2: Mister Zé	10
1.5.3. Entrevista 3: Tiago (Jogador e Proprietário de Viatura)	10
1.5.4. Conclusões das Entrevistas vs. User Stories	10
1.6. Eliciação de requisitos	11
1.6.1. Requisitos não funcionais	13
1.7. Refinamento de requisitos ambíguos	13
1.7.1. Requisitos Funcionais (RF)	14
1.7.2. Requisitos Não Funcionais (RNF)	17
1.8. Especificação do software (SRS – IEEE 830/29148)	21
1.8.1. Introdução	21
1.8.1.1. Propósito	21
1.8.1.2. Âmbito (Scope)	21
1.8.2. Descrição Geral	21
1.8.2.1. Características dos Utilizadores	21
1.8.2.2. Restrições (Constraints)	21
1.8.3. Requisitos Específicos (Conforme ISO/IEC/IEEE 29148)	22
1.8.3.1. Requisitos Funcionais (RF)	22
1.8.3.2. Requisitos Não Funcionais (RNF)	26
1.9. Casos de uso e diagramas UML	28
2. Arquitetura e Design do Software utilizando LLM	37
2.1. Definição da arquitetura global do sistema	37
2.1.1. Identificação da Arquitetura (ISO 42010)	37
2.1.2. Estilo Arquitetural: 3-Layered Architecture (Três Camadas)	38
2.1.2.1. Camada de Apresentação (Presentation Layer - Vue.js)	38
2.1.2.2. Camada de Lógica de Negócio (Business Logic Layer - Flask)	38
2.1.2.3. Camada de Dados (Data Layer - PostgreSQL)	38
2.1.3. Vista de Infraestrutura e Implementação (Physical View)	38
2.1.4. Justificação das Decisões de Design (Contexto vs. Tecnologia)	38
2.1.5. Considerações de Sommerville (Segurança e Disponibilidade)	39
2.2. Revisão arquitetural	39
2.2.1. Revisão Arquitetural: Questionamento e Justificação	39
2.2.1.1. O Raspberry Pi como Servidor de Produção	39
2.2.1.2. Cloudflare Tunnel vs. Port Forwarding	40
2.2.1.3. Vue.js (CSR) em dispositivos antigos	40
2.2.2. Política e Processo de Backup da Base de Dados	40
2.2.2.1. Análise de Requisitos de Backup	40
2.2.2.2. Esquema do Processo de Backup	40
2.2.2.3. Recuperação (Disaster Recovery)	41

2.2.3.	Conclusão da Revisão	41
2.3.	Diagramas de classes	41
2.4.	Diagramas de sequência e componentes necessários	45
2.4.1.	Diagrama de sequência para Autenticar no Sistema	45
2.4.2.	Diagrama de sequência para Registrar Novo Jogador	46
2.4.3.	Diagrama de sequência para Criar Evento	47
2.4.4.	Diagrama de sequência para Publicar Comunicado	47
2.4.5.	Diagrama de sequência para Registrar Presenças em Treino	48
2.4.6.	Diagrama de sequência para Efetuar Convocatória	49
2.4.7.	Diagrama de sequência para Responder a Convocatória	49
2.4.8.	Diagrama de sequência para Registrar Resultado do Jogo	50
2.4.9.	Diagrama de sequência para Disponibilizar Viatura para Boleia	50
2.4.10.	Diagrama de sequência para Reservar Lugar em Viatura	51
2.4.11.	Diagrama de sequência para Cancelar Reserva de Boleia	52
2.5.	Design de interfaces	53
2.6.	Especificação de API e as decisões arquiteturais tomadas	58
2.6.1.	Subsistemas e Endpoints	58
2.6.1.1.	Gestão de Utilizadores	58
2.6.1.2.	Calendário e Eventos	59
2.6.1.3.	Operações Desportivas	59
2.6.1.4.	Logística (Boleias)	59
2.6.2.	Códigos de Estado HTTP	59
2.6.3.	Notas de Implementação	59
2.6.4.	Decisões arquiteturais	59
2.6.4.1.	Modelo Arquitetural: 3-Layered Architecture (Abordagem “uminho”)	60
2.6.4.2.	Padrão de Fachada (Facade Pattern) e Subsistemas	60
2.6.4.3.	Camada de Dados: Padrão DAO (Data Access Object)	60
2.6.4.4.	Escolha Tecnológica: Python (Flask) e Vue.js (CSR)	60
2.6.4.5.	Infraestrutura: Raspberry Pi + Cloudflare Tunnel	61
2.6.4.6.	Política de Backup e Continuidade	61
2.6.4.7.	Conclusão das Decisões	61
3.	Implementação e Desenvolvimento Assistido por LLM	62
3.1.	Configuração do ambiente de desenvolvimento	62
3.2.	Implementação incremental e modular do Subsistema de Eventos	62
3.3.	Geração de código assistida e revisão automática	63
3.4.	Refatoração e deteção de smells	63
4.	Conclusões e Trabalho Futuro	64
	Referências	65
	Lista de Siglas e Acrónimos	66
	Anexos	67
	Bibliografia	68
	Anexo 1 Logo da Universidade do Minho	67

Lista de Figuras

Figura 1	Modelo de domínio	4
Figura 2		35
Figura 3		35
Figura 4		36
Figura 5		36
Figura 6	Diagrama de classes para o subsistema de utilizadores	42
Figura 7	Diagrama de classes para abstração de classes DAO de objetos	43
Figura 8	Diagrama de classes para subsistema de eventos	43
Figura 9		44
Figura 10		45
Figura 11	Diagrama de sequencia para autenticar no sistema	46
Figura 12	Diagrama de sequencia para registar novo jogador	46
Figura 13	Diagrama de sequencia para criar evento	47
Figura 14	Diagrama de sequencia para publicar comunicado	48
Figura 15	Diagrama de sequencia para registar presenças em treino	48
Figura 16	Diagrama de sequencia para efetuar convocatoria	49
Figura 17	Diagrama de sequencia para responder a convocatoria	50
Figura 18	Diagrama de sequencia para registar resultado do jogo	50
Figura 19	Diagrama de sequencia para disponibilizar viatura para boleia	51
Figura 20	Diagrama de sequencia para reservar lugar em viatura	52
Figura 21	Diagrama de sequencia para cancelar reserva de boleia	52
Figura 22	Diagrama de componentes para do sistema	53
Figura 23		54
Figura 24		54
Figura 25		55
Figura 26		55
Figura 27		56
Figura 28		56
Figura 29		57
Figura 30		57
Figura 31		58
Figura 32		58

Lista de Tabelas

Tabela 1	Gestão Centralizada de Jogadores	11
Tabela 2	Mural de Comunicação Unificada	11
Tabela 3	Controlo de Treinos e Assiduidade	12
Tabela 4	Processo de Convocatória Digital	12
Tabela 5	Gestão de Boleias e Viaturas	12
Tabela 6	Usabilidade e Acessibilidade	13
Tabela 7	Fiabilidade e Restrições de Custo	13
Tabela 8	RF01 - Registo de Jogadores	14
Tabela 9	RF02 - Consulta e Atualização de Jogadores	14
Tabela 10	RF03 - Criação de Treinos	15
Tabela 11	RF04 - Registo de Presenças	15
Tabela 12	RF05 - Criação de Convocatória	15
Tabela 13	RF06 - Resposta à Convocatória	15
Tabela 14	RF07 - Oferta de Boleia	16
Tabela 15	RF08 - Reserva de Lugar em Boleia	16
Tabela 16	RF09 - Publicação de Comunicados	16
Tabela 17	RF10 - Autenticação	16
Tabela 18	RNF01 - Simplicidade de Interface	17
Tabela 19	RNF02 - Otimização Móvel	17
Tabela 20	RNF03 - Proteção de Dados	17
Tabela 21	RNF04 - Eficiência de Custos	17
Tabela 22	RNF05 - Integridade Logística	18
Tabela 23	RF11 - Remoção de Utilizadores	18
Tabela 24	RF12 - Gestão de Contactos de Emergência	19
Tabela 25	RF13 - Gestão de Alterações de Última Hora	19
Tabela 26	RF14 - Gestão de Ativos de Transporte	19
Tabela 27	RF15 - Registo Histórico de Resultados	20
Tabela 28	RF01 - Registo de Jogadores	22
Tabela 29	RF02 - Consulta e Atualização de Jogadores	22
Tabela 30	RF03 - Criação de Treinos	22
Tabela 31	RF04 - Registo de Presenças	23
Tabela 32	RF05 - Criação de Convocatória	23
Tabela 33	RF06 - Resposta à Convocatória	23
Tabela 34	RF07 - Oferta de Boleia	23
Tabela 35	RF08 - Reserva de Lugar em Boleia	24
Tabela 36	RF09 - Publicação de Comunicados	24
Tabela 37	RF10 - Leitura de Comunicados	24
Tabela 38	RF11 - Autenticação	24
Tabela 39	RF12 - Remoção de Utilizadores	25
Tabela 40	RF13 - Gestão de Contactos de Emergência	25
Tabela 41	RF14 - Gestão de Alterações de Última Hora	25
Tabela 42	RF15 - Gestão de Ativos de Transporte	26
Tabela 43	RF16 - Registo Histórico de Resultados	26
Tabela 44	RNF01 - Simplicidade de Interface	26
Tabela 45	RNF02 - Otimização Móvel	27

Tabela 46	RNF03 - Proteção de Dados	27
Tabela 47	RNF04 - Eficiência de Custos	27
Tabela 48	RNF05 - Integridade Logística	27

1. Conceção e Engenharia de Requisitos Assistida por LLM

1.1. Análise de contexto e restrições

É necessária a geração de um contexto e de restrições para um o assistente de código gerou o output seguinte para o respetivo prompt:

Prompt:

“Fazes parte de uma equipa de desenvolvimento de sistemas de software, que pretende desenvolver um sistema de software para um clube de futebol português, pequeno e provinciano, com problemas simples que precisam de ser resolvidos pelo sistema. Este clube tem os problemas seguintes:

Problema

Num clube pequeno, a informação dos jogadores costuma estar:

em folhas de papel em Excel disperso ou apenas na cabeça do treinador Isso causa problemas como:

dificuldade em saber quem está inscrito dados desatualizados (contactos, idade, posição)

Problema

Nos clubes pequenos:

datas de jogos ficam em papéis ou mensagens jogadores esquecem-se difícil ver próximos jogos

dificuldade em saber quem pode jogar num jogo

Problema

Normalmente:

convocatórias são feitas no WhatsApp o treinador não sabe quem vai aparecer presença em treinos não fica registada

Problema

A comunicação é muitas vezes:

dispersa por vários grupos de WhatsApp jogadores não recebem informações importantes mudanças de horário perdem-se

Problema

Nos clubes pequenos, os jogos fora implicam:

jogadores a ir em carros de pais ou dirigentes dificuldade em saber quem tem transporte jogadores que ficam sem boleia Tudo é combinado em mensagens ou à última hora.

Gera o contexto para este clube, criando um nome, um dono que seja um apaixonado do futebol, e uma breve história do clube, onde a história não mais de 10 linhas. No contexto, devem estar presentes a descrição dos problemas e as restrições do mesmo.”

OUTPUT

O Contexto do Projeto: Sport Clube “Os Beirões”

O **Sport Clube “Os Beirões”** é o coração pulsante da Vila de São Vicente da Beira. Fundado em 1948, o clube sobrevive da carolice e da mística local. O “dono” (presidente e faz-tudo) é o **Sr. Joaquim “Quim” Barrela**, um antigo guarda-redes que vive para o clube e financia as bolas do seu próprio bolso.

Atualmente, o clube vive um caos logístico: a base de dados dos jogadores resume-se a um caderno sebento no bolso do Quim e a três ficheiros Excel em computadores diferentes. O treinador, o Mister Zé, gere as convocatórias por “instinto” e via WhatsApp, o que resulta frequentemente em jogadores que não aparecem ou que ficam esquecidos na berma da estrada por falta de boleia nos jogos fora.

A História do Clube

“Com mais de 75 anos de história, “Os Beirões” são conhecidos pela sua mística e pelo melhor cozido à portuguesa servido após os jogos. O clube viveu o seu auge nos anos 80, chegando à III Divisão Nacional, mas hoje luta bravamente nos distritais com um plantel de jovens da terra e veteranos resistentes.

O campo, de relvado sintético já algo gasto, é o ponto de encontro da vila aos domingos. O Sr. Quim herdou a presidência do pai e prometeu, antes de se reformar, que traria “ordem e modernidade” àquela casa, garantindo que nenhum jovem da vila ficasse sem jogar por desorganização da secretaria.”

Diagnóstico de Problemas e Restrições

Para o desenvolvimento do nosso sistema, devemos focar-nos nos seguintes pontos críticos extraídos da realidade do clube:

Área Crítica	Descrição do Problema no S.C. Beirões
Gestão de Atletas	Informação dispersa (papel/Excel/memória). Dados como contactos e validade de inscrições estão constantemente desatualizados.
Calendário	Datas de jogos perdem-se em mensagens de telemóvel; jogadores falham compromissos por desconhecimento do calendário.
Operação de Jogo	Dificuldade em gerir convocatórias e registar presenças em treinos; o Mister nunca sabe ao certo com quem conta até ao apito inicial.
Comunicação	Ruído excessivo no WhatsApp; informações cruciais (mudanças de hora) perdem-se no fluxo de conversas informais.
Logística de Transporte	“Caos das Boleias” em jogos fora; dependência de carros particulares de pais/dirigentes sem uma coordenação centralizada.

Restrições do Sistema

1. **Baixa Literacia Digital:** O sistema deve ser extremamente intuitivo para que o Sr. Quim e o Mister Zé o consigam utilizar sem formação avançada.
2. **Custo Reduzido:** Sendo um clube de província, o orçamento para infraestrutura tecnológica (servidores, licenças caras) é quase nulo.
3. **Acesso Mobile:** A solução deve funcionar bem em dispositivos móveis, pois é no campo e no balneário que a maioria das ações ocorre.
4. **Resiliência de Dados:** Garantir que a “cabeça do treinador” deixe de ser o único backup de informação do clube.

Após analisar o output do agente, verificamos que especificamos pouco o prompt, pois os problemas foram pouco desenvolvidos. Por isso pedimos uma revisão através do seguinte prompt:

Prompt

“Desenvolveste pouco os problemas. Adota uma forma mais descritiva, mas sobre o contexto que tens até agora. Deves desenvolver mais o contexto, com sensivelmente uma página, não utilizando o formato de tabelas ou bullet points. Nesse contexto, devem ser mencionados, de forma lógica e relatando os problemas durante a descrição do contexto. Não deves alterar as restrições pois parecem adequadas.”

Output

“O Sport Clube “Os Beirões” é mais do que uma agremiação desportiva; é o último reduto de identidade da Vila de São Vicente da Beira. No centro desta instituição está o Sr. Joaquim “Quim” Barrela, um homem cuja vida se confunde com as linhas de cal do campo. O Sr. Quim é o presidente, o tesoureiro e, muitas vezes, o próprio jardineiro, movido por uma paixão hereditária que o faz gastar a reforma em bolas e

equipamentos. No entanto, o romantismo de “antigamente” colide hoje com uma desorganização que ameaça a sobrevivência do clube. O dia a dia n’Os Beirões é um labirinto de papelada e esquecimentos. A gestão dos jogadores é o primeiro grande obstáculo: a ficha de inscrição de um defesa central pode estar num dossier poeirento na sede, enquanto o contacto de urgência do novo guarda-redes está apenas gravado na memória volátil do Mister Zé, o treinador. Esta dispersão entre folhas soltas, ficheiros Excel guardados em pens perdidas e o “passa-palavra” faz com que, frequentemente, o clube não saiba ao certo quem está legalmente inscrito para o jogo de domingo, resultando em dados de idades e posições constantemente desatualizados que baralham a secretaria.

A vida desportiva da vila sofre com uma névoa de incerteza quanto ao calendário. Sem uma plataforma central, as datas dos jogos e dos treinos flutuam em mensagens de WhatsApp que se perdem no meio de piadas e conversas paralelas. Não é raro um jogador aparecer no campo à hora errada ou, pior, esquecer-se completamente de um compromisso porque a informação ficou enterrada num grupo de chat silenciado. O Mister Zé vive num estado de ansiedade permanente, pois a preparação tática é impossível quando não se sabe quem vai aparecer no treino. As convocatórias são um exercício de adivinhação: o treinador lança os nomes no grupo, mas as respostas são lentas ou inexistentes, e a ausência de um registo de presenças fidedigno impede qualquer avaliação do rendimento ou da assiduidade do plantel ao longo da época.

Esta comunicação fragmentada cria um fosso entre a direção e os atletas. Informações vitais, como a alteração da hora de um treino devido à chuva ou um aviso da associação de futebol, perdem-se no ruído digital, deixando jogadores à deriva. Mas o problema atinge o seu auge logístico nos fins de semana de jogos fora, nas estradas sinuosas da Beira. O clube não tem autocarro próprio, dependendo inteiramente da boa vontade dos dirigentes e dos jogadores. O processo de organizar as boleias é um caos de última hora: combina-se quem leva quem em cima da hora de partida, o que resulta invariavelmente em carros sobrelotados enquanto outros seguem vazios, ou, no pior dos cenários, em jogadores que ficam literalmente a pé na praça da vila porque se assumiu que “alguém” lhes daria boleia. Tudo é gerido no momento, sem uma lista clara de lugares disponíveis ou necessidades de transporte, transformando o que devia ser uma viagem de equipa num pesadelo de coordenação que desgasta a paciência do Sr. Quim e a motivação dos atletas.

Este cenário de “carolice” levada ao limite define as fronteiras do nosso projeto. O sistema a desenvolver não pode ser apenas uma ferramenta técnica; tem de ser o novo “caderno do Quim”, mas digital e partilhado. As restrições são claras e intransigentes: a solução deve ser extremamente simples, capaz de sobreviver à baixa literacia digital dos seus utilizadores principais e ao orçamento inexistente do clube, focando-se na mobilidade e na centralização total da informação para que a mística d’Os Beirões não se perca por falta de organização.”

Análise

Após uma revisão, acreditamos que o contexto agora proposto suficiente para prosseguir com o processo porque explica o background do clube, elucidando alguns problemas que este apresenta, sendo que serão utilizados nos próximos processos.

Portanto, optamos por utilizar este contexto e restrições.

1.2. Definição do domínio do sistema

Com o intuito de possuir um vocabulário universal para tratar as entidades, explicar ou evidenciar as suas relações dentro do sistema, e ainda definir, ainda que com pouco detalhe, algumas restrições em relação às multiplicidades de instâncias das entidades nas relações dentro do sistema.

Por isso, procedemos a uma primeira proposta feita pelo agente, através do seguinte prompt:

Prompt

Com base no contexto selecionado, gera um modelo de domínio que siga o método proposto pelo Prof. António Nestor Ribeiro. Este modelo deve evidenciar as entidades e definir as relações entre elas seguir as restrições, assim como as multiplicidades entre elas.

Output

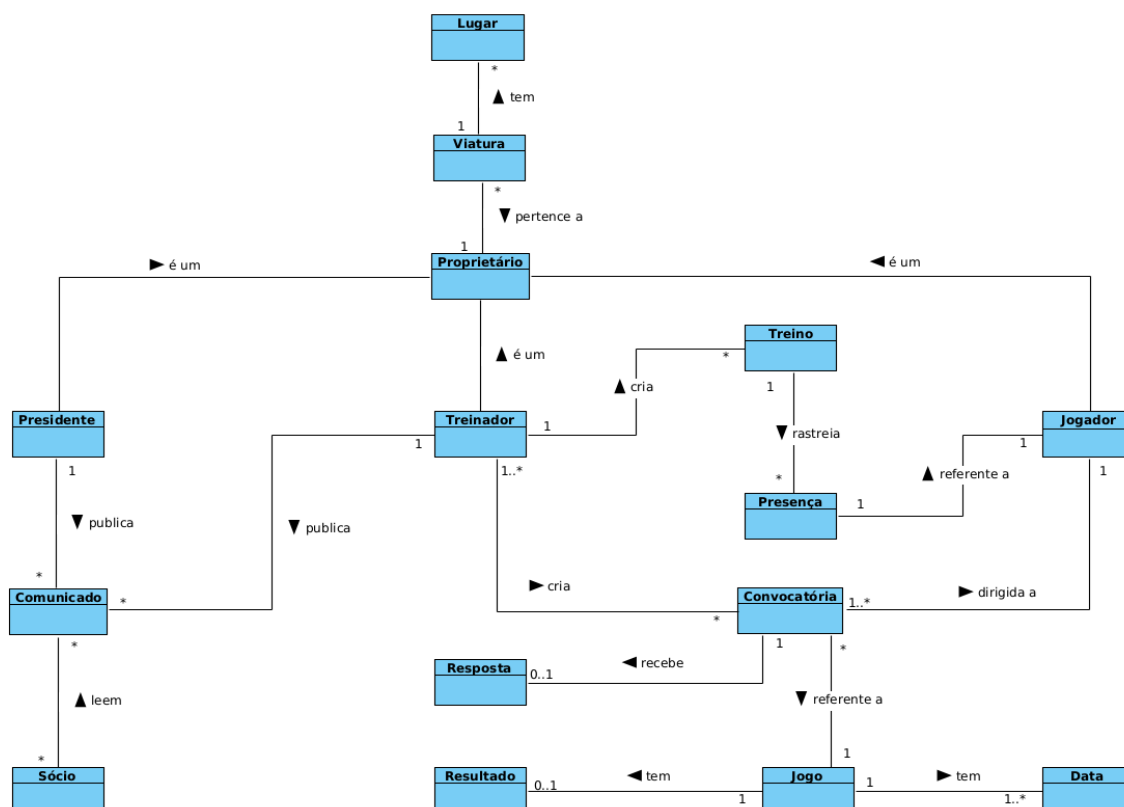


Figura 1: Modelo de domínio

Após pequenas alterações de nomes de relações mais explicativos, como descrito no método que decidimos adotar, decidimos que esta proposta de modelo do agente vai de encontro ao expectável, porque evidencia as entidades presentes no sistema, com as relações que tornam o sistema coeso e robusto.

Prompt

Para o contexto anterior, descreve o modelo de domínio em forma de texto. Não faça mais do que evidenciar o modelo. Deves criar a resposta recorrendo apenas a texto. Não utilizes títulos ou pontos.

Output

O modelo de domínio estrutura-se a partir da entidade **Proprietário**, que serve como base para as especializações **Dirigente**, **Treinador** e **Jogador**, permitindo que qualquer um destes intervenientes possua **Viaturas**. Cada **Viatura** está associada a vários **Lugares** e tem a capacidade de disponibilizar uma **Boleia**. No que respeita à gestão de informação, tanto o **Dirigente** como o **Treinador** podem publicar **Comunicados** para o clube. O **Treinador** é responsável pela criação dos **Treinos**, os quais permitem rastrear as **Presenças** individuais de cada **Jogador**. Além disso, o **Treinador** elabora a **Convocatória**, que é dirigida a um **Jogador** e se refere a um **Jogo** específico, podendo cada convocatória receber uma **Resposta**. O **Jogo** é definido por

uma ou mais **Datas** e pode registrar um **Resultado** associado. **Para** coordenar a logística das viagens, o **Jogo** requer a organização de **Boleias**, sendo que cada **Boleia** é providenciada por uma **Viatura** e transporta os **Jogadores** até ao local da partida.

1.3. Identificação de stakeholders

Para uma definição dos stakeholders para este projeto, propusemos o seguinte prompt, ao qual se deu o seguinte output:

Prompt

Para saber quais são os indivíduos, grupos ou organizações que serão afetadas pelo outcome do projeto, é necessário, agora, identificar os stakeholders para este projeto.

Deves basear-te no livro Software Engineering, de Ian Sommerville para raciocínio, a partir deste ponto.

Output

Com base no contexto do Sport Clube “Os Beirões”, no modelo de domínio apresentado e seguindo a definição de Ian Sommerville — que identifica stakeholders como quaisquer indivíduos ou organizações que sejam afetados pelo sistema ou que tenham uma influência direta nos seus requisitos — identifiquei os seguintes grupos-chave:

1. Sr. Joaquim “Quim” Barrela (Presidente e Tesoureiro)

Ligação: É o principal patrocinador e decisor. Atua como o “dono” do processo atual (o caderno físico) e é o responsável final pela sobrevivência do clube.

Impacto: Será o mais beneficiado pela centralização da informação. O sistema reduzirá o seu desgaste pessoal e garantirá que o investimento da sua reforma seja gerido de forma mais eficiente, evitando coimas por inscrições irregulares e garantindo a continuidade da “mística” do clube.

1. Mister Zé (Treinador)

Ligação: É o utilizador operacional principal. No modelo de domínio, ele é quem cria treinos, convocatórias e comunicações.

Impacto: Terá uma redução direta na sua ansiedade profissional. O sistema permite-lhe passar de uma “adivinhação” para um planeamento tático real, ao ter acesso imediato às presenças e respostas dos jogadores, eliminando o ruído das mensagens de WhatsApp.

1. Jogadores (Atletas)

Ligação: São os utilizadores finais que alimentam o sistema com dados (respostas a convocatórias e necessidades de transporte) e consomem a informação de calendário.

Impacto: Serão afetados positivamente na sua experiência logística e motivacional. A transparência nas datas e, crucialmente, a organização das boleias, garante que nenhum jogador fique “a pé”, aumentando o compromisso com a equipa.

1. Dirigentes e Proprietários de Viaturas

Ligação: Stakeholders operacionais e logísticos. São eles que disponibilizam os recursos (carros) para viabilizar a atividade do clube fora de casa.

Impacto: Serão afetados pela otimização de recursos. O sistema evitará que uns circulem sobrelotados enquanto outros vão vazios, formalizando a sua contribuição e facilitando a comunicação de avisos da associação através de comunicados oficiais.

1. Associação de Futebol (Entidade Reguladora)

Ligação: Stakeholder externo (conforme a classificação de Sommerville). Define as regras de inscrição e os calendários oficiais.

Impacto: Embora não use o sistema diretamente, é afetada pela qualidade dos dados. Um sistema organizado garante que o clube cumpra os prazos legais e apresente fichas de jogo atualizadas, reduzindo problemas burocráticos e administrativos para a associação.

Analisando o output do agente, determinamos que, apesar de ser um output correto, não engloba os desenvolvedores do projeto como stakeholder do projeto, o que nos levou a propor o seguinte prompt:

Prompt

Como os condutores serão restritos aos utilizadores internos diretos, não acredito que deva existir mais dirigentes e voluntários. Deves eliminá-los da definição, e deves colocar ainda a equipa de desenvolvimento. Além disso, deves separar entre stakeholders internos, externos e stakeholders no processo de desenvolvimento.

Output

1. Stakeholders Internos (Operacionais e de Gestão)

- **Sr. Joaquim “Quim” Barrela (Presidente/Dono):** É o principal patrocinador do projeto. O seu interesse é centralizar a informação (“ordem e modernidade”) para garantir que o clube sobreviva e funcione sem a sua intervenção constante em detalhes logísticos.
- **Mister Zé (Treinador):** É o utilizador principal para as decisões técnicas. Atualmente, depende da sua memória e do WhatsApp. O seu interesse é saber com quem pode contar para treinos e jogos de forma fiável.
- **Jogadores (Atletas):** São os utilizadores finais que consomem a informação de calendários, convocações e logística de transporte. São afetados diretamente pela melhoria da comunicação e pela organização das boleias.
- **Equipa de Secretaria/Administrativa (Mesmo que informal):** Qualquer pessoa que ajude o Sr. Quim com as fichas de papel ou ficheiros Excel. Estes stakeholders serão afetados pela transição do papel/digital disperso para um sistema único.

2. Stakeholders Externos

- **Associação de Futebol Local/Regional:** Entidade reguladora que define prazos de inscrição e calendários oficiais. O sistema deve garantir que o clube cumpre as normas desta entidade.

3. Stakeholders do Processo de Desenvolvimento

- **Equipa de Desenvolvimento de Software (Nós):** Responsáveis por transformar os problemas descritos (caos logístico e de informação) numa solução técnica que respeite as restrições de baixa literacia digital e custo reduzido.

Analisando o output do agente, entendeu-se que a sua proposta vai de encontro ao que planeamos como resposta à definição dos stakeholders deste projeto, onde se enquadra aqueles com impacto direto como o presidente e o treinador, assim como o stakeholder indireto que é a entidade reguladora que atua sobre o clube. Por fim, a equipa de desenvolvimento, já que será influenciada também pelo resultado do projeto, direta ou indiretamente.

1.4. Geração de user stories

Passando agora a uma geração de user stories, que ilustrem os problemas vividos por parte de stakeholders com envolvimento direto no produto/projeto, torna-se numa forma leve de começar o levantamento de requisitos do sistema.

Com isto em mente, após informar o agente das alterações feitas ao prompt anterior, propusemos o seguinte **prompt**:

Prompt

Baseando nas alterações efetuadas, procede à geração de user stories que ilustrem os problemas sobre os quais vamos trabalhar. Para isso, utiliza como referência o livro *Software Engineering* de Ian Sommerville.

Output

Complementarmente, seguiremos as diretrizes da norma ISO/IEC/IEEE 29148:2018 (que substituiu o antigo padrão IEEE 830) para a engenharia de requisitos, garantindo que cada história seja Verificável, Necessária e Priorizada.

1. Stakeholder: Sr. Joaquim “Quim” Barrela (Presidente/Administrador)

O foco aqui é a transição do “caderno” para o digital, combatendo a dispersão da informação.

User Story [US01] - Centralização de Fichas:

- História: Enquanto Presidente, eu quero ter uma lista digital única e atualizada de todos os jogadores (nome, idade, posição e contacto), para que eu saiba exatamente quem está legalmente apto para os jogos de domingo sem procurar em dossiers de papel.
- Justificação: Resolve o problema da “ficha de inscrição num dossier poeirento” e da “memória volátil do Mister Zé”. Garante que a secretaria não se baralhe com dados desatualizados.
- Critério de Aceitação: O sistema deve permitir visualizar a lista completa de jogadores em menos de 3 cliques no telemóvel.

User Story [US02] - Comunicados Oficiais:

- História: Enquanto Presidente, eu quero publicar avisos oficiais no sistema, para que a informação vital (como avisos da associação) não se perca no ruído e nas piadas dos grupos de WhatsApp.
- Justificação: Combate o “fosso entre a direção e os atletas” e garante que informações críticas cheguem a todos sem interferência de conversas paralelas.
- Critério de Aceitação: O comunicado deve aparecer com destaque imediato na página inicial de todos os utilizadores.

2. Stakeholder: Mister Zé (Treinador)

O foco é a redução da ansiedade através do planeamento e controlo de assiduidade.

User Story [US03] - Gestão de Treinos e Presenças:

- História: Enquanto Treinador, eu quero registar as datas dos treinos e marcar quem esteve presente, para que eu possa avaliar o rendimento e a assiduidade do plantel ao longo da época de forma justa.
- Justificação: Resolve a impossibilidade de preparação tática por não se saber quem aparece no treino. Substitui o “registo inexistente” por um histórico fidedigno.
- Critério de Aceitação: O registo de presenças para um treino deve ser feito numa única lista de verificação (checkboxes) rápida.

User Story [US04] - Convocatória e Resposta Direta:

- História: Enquanto Treinador, eu quero lançar a convocatória para o jogo e receber uma confirmação (Sim/Não) de cada jogador, para que eu saiba com antecedência quem estará disponível para o jogo.
- Justificação: Elimina o “exercício de adivinhação” e as respostas lentas ou inexistentes no chat do WhatsApp.
- Critério de Aceitação: O sistema deve mostrar um contador em tempo real: “X jogadores confirmados / Y convocados”.

3. Stakeholder: Jogadores (Atletas)

O foco é a mobilidade e a resolução do caos logístico.

User Story [US05] - Consulta de Calendário:

- História: Enquanto Jogador, eu quero consultar as datas e horas exatas dos jogos e treinos no meu telemóvel, para que eu não apareça à hora errada nem me esqueça dos compromissos.
- Justificação: Resolve a “névoa de incerteza” e as mensagens perdidas no chat silenciado.
- Critério de Aceitação: O calendário deve distinguir claramente entre “Treino” e “Jogo”.

User Story [US06] - Coordenação de Boleias (O “Pesadelo Logístico”):

- História: Enquanto Jogador, eu quero ver quais as viaturas disponíveis para os jogos fora e reservar um lugar, para que eu não fique “a pé” na praça da vila no momento da partida.
- Justificação: Ataca o ponto fulcral do problema: o caos de última hora e os carros sobrelotados vs. carros vazios nas estradas sinuosas da Beira.
- Critério de Aceitação: Deve ser visível o número de lugares vagos em cada viatura de um dirigente ou colega.

4. Stakeholder: Proprietário de Viatura (Dirigente/Jogador)

O foco é a otimização de recursos e organização da viagem.

User Story [US07] - Disponibilização de Transporte:

- História: Enquanto Proprietário, eu quero indicar quantos lugares tenho disponíveis no meu carro para um jogo específico, para que a equipa possa organizar o transporte de forma equilibrada.
- Justificação: Permite ao Sr. Quim e aos outros condutores saberem exatamente a capacidade de transporte total para a deslocação, evitando o stress de última hora.
- Critério de Aceitação: O proprietário deve poder editar a sua oferta de lugares até à hora de partida.

Resumo Técnico da Abordagem

Estas *User Stories* seguem o princípio INVEST (Independent, Negotiable, Valuable, Estimable, Small, Testable), frequentemente citado na engenharia de software moderna para garantir que cada funcionalidade é passível de ser desenvolvida de forma ágil, respeitando a restrição de literacia digital reduzida através de interações simples (Sim/Não, Checklists, Listas Visuais).

Dado que a proposta do agente segue a norma ISO 29148 proposta pelo IEEE, e analisando a sua concordância com o contexto até agora desenvolvido, concordamos que as *user stories* irão suportar a engenharia de requisitos restante.

1.5. Simulação de entrevistas com stakeholders

Para possuímos uma base melhor de eliciação de requisitos, simulamos agora entrevistas com os stakeholders, no formato de pergunta/entrevista aberta, dado que pretendemos retirar os problemas. Para isso, utilizamos o seguinte prompt:

Prompt

Com base no contexto até agora desenvolvido, deves simular entrevistas com os stakeholders.
Estas devem suportar as User Stories e devem concordar com o contexto até agora desenvolvido.

Output

1.5.1. Entrevista 1: Sr. Joaquim “Quim” Barreira

Papel: Presidente, Tesoureiro e “Pai” do Clube. **Foco:** [US01], [US02]

“Entrevistador: Sr. Quim, como é que o senhor sabe hoje se o defesa central está regularizado para o jogo?

Sr. Quim: Olhe, é um ai-jesus. Tenho de ir à sede, abrir aquele dossier azul que já tem a mola partida e rezar para que a ficha de inscrição esteja lá. Às vezes o Mister Zé diz-me que o rapaz já assinou, mas o papel ficou no carro de outro diretor. Eu gasto a minha reforma em bolas, não posso gastar em multas porque um miúdo jogou sem estar inscrito.

Entrevistador: E quando há um aviso importante da Associação de Futebol?

Sr. Quim: Eu ponho no grupo de WhatsApp, mas logo a seguir alguém manda um vídeo de um golo ou uma piada, e o meu aviso desaparece. Depois dizem-me: “Ah, Sr. Quim, não vi nada!”. Eu precisava de um sítio onde o que eu escrevo ficasse lá pregado, como um edital na praça, mas no telemóvel deles.”

1.5.2. Entrevista 2: Mister Zé

Papel: Treinador (Mister). **Foco:** [US03], [US04]

“Entrevistador: Mister, qual é a sua maior dificuldade à terça-feira, no dia de treino?

Mister Zé: É a incerteza. Eu preparo um exercício tático para dez, e aparecem seis. Ou aparecem doze, mas três esqueceram-se que o treino era às 20h00 e aparecem às 20h30. Não há registo de nada. No fim do mês, o Sr. Quim pergunta-me quem merece jogar e eu tenho de puxar pela cabeça... a memória já falha.

Entrevistador: E as convocatórias para domingo?

Mister Zé: Um pesadelo. Lanço os nomes no grupo e fico a olhar para o ecrã. Uns respondem logo, outros só no sábado à noite. E há sempre aquele que diz “eu respondi!”, mas a mensagem dele ficou perdida no meio de 50 outras. Se eu tivesse um botão de “Sim” ou “Não” para cada um, a minha vida ganhava cinco anos de saúde.”

1.5.3. Entrevista 3: Tiago (Jogador e Proprietário de Viatura)

Papel: Defesa Central e condutor voluntário. **Foco:** [US05], [US06], [US07]

“Entrevistador: Tiago, já alguma vez ficaste “a pé” para um jogo fora?

Tiago: Eu não, porque levo o meu carro, mas já vi colegas meus ficarem na praça da vila a olhar para o ar porque o colega que costuma dar boleia foi direto para o jogo com a namorada e não avisou.

Entrevistador: Como organizam as vagas nos carros?

Tiago: É na base do grito, minutos antes de sair. “Quem tem lugar?”, “Eu tenho dois!”, “Ah, mas eu preciso de três!”. Acabamos por ir uns todos apertados e outros carros com lugares vazios. É um desperdício de gasolina e paciência. Se eu pudesse dizer na aplicação: “Pessoal, saio da praça às 13h e tenho 3 lugares no meu Clio”, escusávamos de andar nesta confusão de última hora.”

1.5.4. Conclusões das Entrevistas vs. User Stories

Insight da Entrevista	User Story Validada	Justificação Técnica
“Dossier azul com mola partida”	“[US01]”	“Necessidade de persistência de dados centralizada e acessível.”
“O meu aviso desaparece no WhatsApp”	“[US02]”	“Necessidade de um canal de comunicação unidirecional de alta prioridade.”
“Puxar pela cabeça... a memória falha”	“[US03]”	“Necessidade de um log de presenças para análise de desempenho.”
“Um botão de Sim ou Não”	“[US04]”	“Simplificação da entrada de dados (Baixa Literacia Digital).”
“Ficar na praça a olhar para o ar”	“[US06] / [US07]”	“Gestão de recursos (lugares em viaturas) para otimização logística.”

As entrevistas confirmam que a tecnologia não deve ser um “obstáculo”, mas um facilitador de processos que já existem (embora de forma caótica). O foco deve ser a **baixa fricção**.

Analisando o output do agente, acreditamos que a simulação de entrevistas aproxima-se do que pretendemos, dado que estas foram especificadas segundo a norma da IEEE.

Com isto, entendemos possuir a base para proceder à eliciação de requisitos.

1.6. Eliciação de requisitos

Passando agora para uma primeira fase de eliciação de requisitos com base nos recursos que possuímos, propusemos o seguinte prompt ao agente:

Prompt

Com base no contexto, procede agora a uma eliciação de requisitos, como descrito no livro de Sommerville.

Estes devem ser devidamente justificados sobre a sua origem e deve ser definida uma prioridade de implementação. Deves identificar os requisitos segundo o formato RF e RNF para requisitos funcionais e não funcionais. O número esperado de requisitos deve se encaixar no intervalo de 20 a 30 requisitos.

Output

Com base no contexto d'Os Beirões e seguindo as diretrizes de Ian Sommerville para a distinção entre requisitos de utilizador (metas abstratas) e requisitos de sistema (descrições detalhadas da funcionalidade), elaborei a seguinte elicitação.

Esta lista foca-se na simplicidade extrema exigida para combater a baixa literacia digital, enquanto resolve o “caos logístico” identificado nas entrevistas.

Requisito de utilizador	O Presidente deve poder centralizar e gerir a informação dos atletas sem recorrer a dossiers físicos.
Fonte	Sr. Quim (Simulação de Entrevista)
Área do sistema	Gestão de Dados Mestres
Requisitos de sistema	1. O sistema deve permitir o registo de Jogadores com Nome, Data de Nascimento, Posição e Contacto de Urgência. 2. O sistema deve permitir o upload de uma fotografia por jogador para facilitar a identificação visual (essencial para baixa literacia). 3. O sistema deve emitir um alerta visual na ficha do jogador caso a data de validade da inscrição na Associação esteja próxima do fim. 4. O sistema deve permitir a exportação da lista de inscritos para um formato simples (PDF/Imagem) para partilha rápida.
Relevância para a aplicação	Elimina o “dossier azul de mola partida” e garante que o Sr. Quim tenha os dados legais sempre à mão.

Tabela 1: Gestão Centralizada de Jogadores

Requisito de utilizador	A direção e o treinador devem poder comunicar avisos oficiais que não se percam em conversas paralelas.
Fonte	Sr. Quim e Mister Zé (Simulação de Entrevista)
Área do sistema	Comunicação Oficial
Requisitos de sistema	1. O sistema deve possuir um mural de “Comunicados” na página inicial, visível imediatamente após o login. 2. Cada comunicado deve ter um título, corpo de texto curto e data de publicação automática. 3. O sistema deve permitir filtrar comunicados por categoria (Direção ou Equipa Técnica).
Relevância para a aplicação	Resolve o problema das informações vitais que ficam enterradas em piadas e vídeos no WhatsApp.

Tabela 2: Mural de Comunicação Unificada

Requisito de utilizador	O Treinador deve poder organizar treinos e controlar quem comparece para avaliar o rendimento.
Fonte	Mister Zé (Simulação de Entrevista)
Área do sistema	Gestão Desportiva
Requisitos de sistema	1. O sistema deve permitir a criação de eventos do tipo “Treino” com data, hora e local. 2. O sistema deve gerar uma lista de presenças automática baseada no plantel ativo. 3. O Treinador deve poder marcar a presença (Visto/Falta) através de um único toque no nome do jogador. 4. O sistema deve calcular a percentagem de assiduidade mensal de cada jogador automaticamente.
Relevância para a aplicação	Permite ao Mister Zé sair do estado de “adivinhação” e planejar exercícios baseados em dados reais de assiduidade.

Tabela 3: Controlo de Treinos e Assiduidade

Requisito de utilizador	O sistema deve resolver o caos das convocatórias e garantir respostas rápidas dos atletas.
Fonte	Mister Zé e Jogadores (Simulação de Entrevista)
Área do sistema	Gestão de Competição
Requisitos de sistema	1. O sistema deve permitir associar uma “Convocatória” a um “Jogo” específico. 2. O Jogador deve visualizar apenas dois botões grandes na sua área de convocatória: “Vou” (Confirmar) ou “Não Vou” (Recusar). 3. O sistema deve permitir ao Jogador adicionar uma nota curta de justificação em caso de recusa. 4. O sistema deve apresentar ao Treinador um resumo: “Confirmados: X Recusados: Y Sem Resposta: Z”.
Relevância para a aplicação	Garante que o Treinador saiba com quem contar para o domingo antes do dia do jogo.

Tabela 4: Processo de Convocatória Digital

Requisito de utilizador	Os utilizadores devem poder organizar o transporte para jogos fora de forma equitativa e antecipada.
Fonte	Tiago e Contexto Logístico
Área do sistema	Logística de Transporte (Boleias)
Requisitos de sistema	1. O sistema deve permitir que Proprietários (Dirigentes/Jogadores) registem a sua Viatura e o número de lugares disponíveis. 2. O sistema deve associar “Boleias” a um “Jogo” específico fora de casa. 3. O sistema deve permitir que um Jogador convocado reserve um lugar vago numa das viaturas listadas. 4. O sistema deve bloquear reservas numa viatura quando a lotação (Lugares) for atingida. 5. O sistema deve gerar um mapa de transporte indicando claramente quem vai em que carro.
Relevância para a aplicação	Elimina o “pesadelo de coordenação” na praça da vila e evita carros sobrelotados ou jogadores esquecidos.

Tabela 5: Gestão de Boleias e Viaturas

1.6.1. Requisitos não funcionais

Requisito de utilizador	A interface deve ser extremamente fácil de usar, mesmo para quem não tem facilidade com tecnologia.
Fonte	Contexto (Baixa Literacia Digital)
Área do sistema	Requisitos Não Funcionais (Usabilidade)
Requisitos de sistema	1. A interface deve utilizar ícones autoexplicativos (ex: uma bola para treinos, um carro para boleias) em conjunto com texto. 2. O sistema deve ser desenhado sob a filosofia “Mobile-First”, garantindo que todos os botões tenham dimensões mínimas de 44x44 pixels para fácil toque. 3. O contraste das cores deve seguir as normas de acessibilidade para facilitar a leitura sob luz solar (no campo).
Relevância para a aplicação	Condição sine qua non para a sobrevivência do projeto num ambiente de “carolice” e baixa literacia.

Tabela 6: Usabilidade e Acessibilidade

Requisito de utilizador	O sistema deve ser gratuito de manter e funcionar de forma fiável.
Fonte	Contexto (Orçamento Inexistente)
Área do sistema	Requisitos Não Funcionais (Operacionais)
Requisitos de sistema	1. O sistema deve ser alojado em infraestrutura de “Free Tier” (ex: Firebase, Vercel ou similar). 2. O tempo de carregamento da página inicial não deve exceder os 3 segundos em ligações 3G/4G (comum nas estradas da Beira). 3. O sistema deve garantir a persistência dos dados (Backups automáticos diários na cloud).
Relevância para a aplicação	Garante que o clube não tenha custos fixos que não pode suportar e que a informação não se perca como os ficheiros Excel em pens perdidas.

Tabela 7: Fiabilidade e Restrições de Custo

Após uma análise do output, é notável que existe ambiguidade nos requisitos do utilizador, que necessita de refinamento. Como irá ser feito no próximo sub capítulo, entendemos que podemos utilizar a resposta como base para prosseguir para o refinamento.

1.7. Refinamento de requisitos ambíguos

Prompt

Com tudo o que foi gerado até agora, deves proceder a uma eliciação de requisitos funcionais e não funcionais, segundo a norma e a descrição de Ian Somerville. Deves dispor no formato de tabela, destilando requisitos de utilizador, e para os quais eliciar requisitos de sistema.

A tabela deve possuir, pelo menos, duas colunas, onde as linhas evidenciam

- Requisito de utilizador
- Fonte
- Área do sistema
- Requisitos do sistema

- Relevância para a aplicação

Caso a norma indique outras informação, deves seguir a norma.

É esperado que seja possível eliciar de 20 a 30 requisitos funcionais e não funcionais, sendo que cada requisito do utilizador deve gerar alguns requisitos de sistema.

Os requisitos não devem possuir ambiguidades, sendo que um requisito de gestão de algo é ambíguo, mas 4 requisitos de CRUD não possuem ambiguidades.

Output

1.7.1. Requisitos Funcionais (RF)

Requisito de utilizador	O Presidente deve ser capaz de registar novos jogadores no sistema.
Fonte	Sr. Quim (Entrevista - Problema dos dossiers poeirentos)
Área do sistema	Gestão de Plantel
Requisitos de sistema	1. O sistema deve disponibilizar um formulário de inserção de dados. 2. O utilizador deve fornecer: Nome Completo, Data de Nascimento, Posição Preferencial e Contacto Telefónico. 3. O sistema deve validar se o campo "Nome" e "Contacto" estão preenchidos antes de guardar. 4. O sistema deve gerar um identificador único para o jogador após o sucesso da operação.
Relevância para a aplicação	Essencial para centralizar a informação que atualmente está dispersa em papéis e dossiers.

Tabela 8: RF01 - Registo de Jogadores

Requisito de utilizador	O Presidente deve poder consultar e editar os dados dos jogadores inscritos.
Fonte	Sr. Quim (Contexto - Dados de idades e posições desatualizados)
Área do sistema	Gestão de Plantel
Requisitos de sistema	1. O sistema deve listar todos os jogadores por ordem alfabética de nome. 2. O sistema deve permitir a alteração de qualquer campo (exceto ID único) de um jogador existente. 3. O sistema deve guardar a data da última atualização de cada ficha de jogador.
Relevância para a aplicação	Garante que a secretaria tenha dados fidedignos para as fichas de jogo de domingo.

Tabela 9: RF02 - Consulta e Atualização de Jogadores

Requisito de utilizador	O Treinador deve poder criar eventos de treino no calendário.
Fonte	Mister Zé (Entrevista - Incerteza no calendário)
Área do sistema	Planeamento Desportivo
Requisitos de sistema	1. O sistema deve permitir inserir a data e a hora de início do treino. 2. O sistema deve permitir seleccionar o local do treino (Campo Principal ou Sintético). 3. O sistema deve impedir a criação de treinos em datas passadas.
Relevância para a aplicação	Combate a “névoa de incerteza” sobre as datas dos treinos que se perdem no WhatsApp.

Tabela 10: RF03 - Criação de Treinos

Requisito de utilizador	O Treinador deve registar as presenças dos jogadores em cada treino.
Fonte	Mister Zé (Contexto - Ausência de registo fidedigno de assiduidade)
Área do sistema	Controlo de Rendimento
Requisitos de sistema	1. Para cada treino criado, o sistema deve apresentar a lista de todos os jogadores ativos. 2. O sistema deve permitir marcar cada jogador como “Presente” ou “Ausente” através de uma caixa de seleção. 3. O sistema deve permitir adicionar uma nota de texto curta (ex: “Lesionado”) ao registo de presença.
Relevância para a aplicação	Permite ao treinador avaliar de forma justa a assiduidade e o compromisso dos atletas.

Tabela 11: RF04 - Registo de Presenças

Requisito de utilizador	O Treinador deve criar convocatórias para jogos oficiais.
Fonte	Mister Zé (Entrevista - “Exercício de adivinhação”)
Área do sistema	Operações de Jogo
Requisitos de sistema	1. O sistema deve permitir seleccionar um Jogo previamente criado. 2. O utilizador deve seleccionar os jogadores a convocar a partir da lista geral de inscritos. 3. O sistema deve enviar uma notificação interna aos jogadores seleccionados.
Relevância para a aplicação	Formaliza o processo de convocatória, retirando-o do ruído das conversas paralelas.

Tabela 12: RF05 - Criação de Convocatória

Requisito de utilizador	O Jogador deve responder à convocatória (Confirmar/Recusar).
Fonte	Atleta Tiago (Entrevista - Botão de Sim ou Não)
Área do sistema	Operações de Jogo
Requisitos de sistema	1. O sistema deve apresentar ao jogador as opções “Vou” e “Não Vou” para cada convocatória pendente. 2. O sistema deve registar a resposta e a hora em que foi efetuada. 3. O sistema deve permitir que o jogador altere a resposta até 12 horas antes do jogo.
Relevância para a aplicação	Dá ao treinador a certeza de quem estará disponível para o jogo com antecedência.

Tabela 13: RF06 - Resposta à Convocatória

Requisito de utilizador	O Proprietário deve disponibilizar lugares na sua viatura para boleia.
Fonte	Contexto (Caos de última hora nas viagens)
Área do sistema	Logística de Transporte
Requisitos de sistema	1. O utilizador deve seleccionar a sua viatura registada e associá-la a um Jogo específico. 2. O utilizador deve indicar o número de lugares disponíveis para transporte (excluindo o condutor). 3. O sistema deve apresentar a lista de lugares como “Vagos”.
Relevância para a aplicação	Ponto central para resolver a desorganização das viagens para jogos fora.

Tabela 14: RF07 - Oferta de Boleia

Requisito de utilizador	O Jogador deve poder reservar um lugar numa viatura disponível.
Fonte	Contexto (Jogadores que ficam “a pé” na praça da vila)
Área do sistema	Logística de Transporte
Requisitos de sistema	1. O sistema deve listar todas as viaturas com lugares vagos para o jogo selecionado. 2. O jogador deve seleccionar uma viatura para efetuar a reserva. 3. O sistema deve decrementar o número de lugares vagos da viatura após a reserva. 4. O sistema deve impedir reservas em viaturas com 0 lugares vagos.
Relevância para a aplicação	Garante que todos os jogadores tenham transporte confirmado antes da hora de partida.

Tabela 15: RF08 - Reserva de Lugar em Boleia

Requisito de utilizador	O Dirigente deve publicar comunicados oficiais para todos os membros.
Fonte	Sr. Quim (Entrevista - Informação perdida no WhatsApp)
Área do sistema	Comunicação Centralizada
Requisitos de sistema	1. O sistema deve disponibilizar um campo de título e um corpo de texto para o comunicado. 2. O sistema deve marcar o comunicado com a data de publicação e o nome do autor. 3. O sistema deve permitir anexar um ficheiro PDF (opcional).
Relevância para a aplicação	Cria um canal oficial de comunicação, eliminando o fosso entre direção e atletas.

Tabela 16: RF09 - Publicação de Comunicados

Requisito de utilizador	Todos os utilizadores devem autenticar-se para aceder ao sistema.
Fonte	Contexto (Necessidade de centralização e segurança de dados)
Área do sistema	Segurança e Acessos
Requisitos de sistema	1. O sistema deve solicitar o contacto telefónico e uma palavra-passe. 2. O sistema deve validar as credenciais contra a base de dados de utilizadores ativos. 3. O sistema deve redireccionar o utilizador para o seu perfil específico (Treinador, Jogador, Dirigente).
Relevância para a aplicação	Protege os dados sensíveis dos jogadores e garante que as operações são feitas pelos perfis corretos.

Tabela 17: RF10 - Autenticação

1.7.2. Requisitos Não Funcionais (RNF)

Requisito de utilizador	O sistema deve ser extremamente fácil de usar por pessoas com pouca experiência digital.
Fonte	Contexto (Baixa literacia digital dos utilizadores principais)
Área do sistema	Usabilidade
Requisitos de sistema	1. A interface deve utilizar ícones grandes e texto com contraste elevado. 2. Todas as ações principais (Confirmar Jogo, Reservar Boleia) não devem exigir mais de 3 toques no ecrã. 3. O sistema deve evitar jargão técnico, usando termos como “Ir ao Jogo” em vez de “Submeter Resposta Boolean”.
Relevância para a aplicação	Crítico para a sobrevivência do projeto; se for complexo, os utilizadores voltarão ao papel.

Tabela 18: RNF01 - Simplicidade de Interface

Requisito de utilizador	O sistema deve funcionar perfeitamente em telemóveis antigos e com pouca internet.
Fonte	Contexto (Foco na mobilidade e orçamento inexistente)
Área do sistema	Desempenho e Disponibilidade
Requisitos de sistema	1. O tamanho total da aplicação/página inicial não deve exceder 2MB. 2. O sistema deve carregar a lista de convocatórias em menos de 3 segundos em redes 3G. 3. O sistema deve ser responsivo para ecrãs pequenos (largura mínima de 320px).
Relevância para a aplicação	Permite que os jogadores consultem informações no campo ou na praça da vila.

Tabela 19: RNF02 - Otimização Móvel

Requisito de utilizador	Os dados dos jogadores (contactos e moradas) devem estar protegidos.
Fonte	Engenharia de Software (Segurança/Privacidade)
Área do sistema	Segurança de Dados
Requisitos de sistema	1. As palavras-passe devem ser armazenadas utilizando algoritmos de cifra (hashing) seguros. 2. Apenas utilizadores com perfil “Presidente” ou “Treinador” podem ver os números de telefone privados dos jogadores.
Relevância para a aplicação	Cumprimento ético e legal (RGPD), evitando a exposição indevida de dados pessoais.

Tabela 20: RNF03 - Proteção de Dados

Requisito de utilizador	O custo de manutenção do sistema deve ser nulo ou residual.
Fonte	Contexto (Orçamento inexistente do clube)
Área do sistema	Arquitetura
Requisitos de sistema	1. O sistema deve ser alojado em serviços de “Free Tier” (ex: Firebase, Vercel ou Supabase). 2. O sistema deve utilizar uma base de dados que não gere custos por volumes de dados inferiores a 1GB.
Relevância para a aplicação	Inviabiliza o projeto se o Sr. Quim tiver de pagar mensalidades elevadas.

Tabela 21: RNF04 - Eficiência de Custos

Requisito de utilizador	A informação sobre as boleias deve estar sempre atualizada em tempo real.
Fonte	Contexto (Evitar carros sobrelotados ou jogadores esquecidos)
Área do sistema	Confiabilidade
Requisitos de sistema	1. Sempre que um lugar é reservado, o sistema deve atualizar a contagem global imediatamente (Sincronização em tempo real). 2. Se dois jogadores tentarem reservar o último lugar ao mesmo tempo, o sistema deve garantir que apenas o primeiro pedido é processado (Controlo de concorrência).
Relevância para a aplicação	Evita falhas logísticas graves no momento da partida para os jogos.

Tabela 22: RNF05 - Integridade Logística

Nota Final: Esta eliciação cobre os processos críticos de gestão de plantel, logística de transporte e planeamento desportivo, transformando os problemas de “carolice” numa estrutura digital sustentável.

Analisando a resposta do agente, a equipa concordou que está em falta um requisito funcional para remoção de utilizadores, caso alguém queira sair do clube. Por isso, propusemos o seguinte prompt.

Prompt

Está em falta um requisito funcional para a remoção de um utilizador do sistema. Deves adicionar e rever outros requisitos que estejam em falta, justificando cada um e indicando a fonte de derivação, que apenas deve advir do contexto até então desenvolvido

Output

Para completar a eliciação e garantir que o sistema cobre todas as lacunas identificadas no contexto — especialmente a segurança operacional e a gestão de imprevistos — adicionei o requisito de remoção solicitado e derivei outros quatro requisitos críticos que estavam em falta.

As fontes de derivação focam-se nos problemas específicos do texto: o contacto de urgência “volátil”, as alterações por causa da chuva e a gestão das viaturas privadas.

Novos Requisitos Funcionais (RF)

Requisito de utilizador	O Presidente deve poder remover utilizadores que já não pertençam ao clube.
Fonte	Derivação Lógica (Contexto: “Inscrição num dossier poeirento” implica necessidade de limpeza).
Área do sistema	Gestão de Utilizadores
Requisitos de sistema	1. O sistema deve permitir ao administrador selecionar um utilizador e marcar a conta como “Inativa”. 2. O sistema deve impedir que utilizadores removidos/inativos iniciem sessão. 3. O sistema deve manter os dados históricos (presenças e resultados) associados ao utilizador para fins estatísticos, mas ocultá-lo das listas de convocatórias e transporte.
Relevância para a aplicação	Evita que a base de dados fique “poeirenta” e desorganizada com membros que já abandonaram a vila ou o clube.

Tabela 23: RF11 - Remoção de Utilizadores

Requisito de utilizador	O Treinador e o Presidente devem ter acesso imediato ao contacto de urgência de cada jogador.
Fonte	Contexto (“Contacto de urgência do novo guarda-redes está apenas gravado na memória volátil do Mister Zé”).
Area do sistema	Segurança e Saúde
Requisitos de sistema	1. O formulário de registo de jogador deve incluir obrigatoriamente um campo “Contacto de Emergência” (Nome e Telefone). 2. O sistema deve exibir um botão de “Chamada de Emergência” no perfil de cada jogador, visível apenas para a Direção e Treinador. 3. Ao clicar no botão, o telemóvel do utilizador deve iniciar automaticamente a chamada para o contacto registado.
Relevância para a aplicação	Resolve diretamente a falha de segurança mencionada no contexto, garantindo que em caso de lesão a informação não dependa da memória de ninguém.

Tabela 24: RF12 - Gestão de Contactos de Emergência

Requisito de utilizador	O Treinador deve poder cancelar ou alterar a hora de eventos (treinos/jogos) de forma rápida.
Fonte	Contexto (“Alteração da hora de um treino devido à chuva... deixa jogadores à deriva”).
Area do sistema	Planeamento Desportivo
Requisitos de sistema	1. O sistema deve permitir editar a data, hora ou estado (Ativo/Cancelado) de qualquer evento criado. 2. Ao alterar ou cancelar um evento, o sistema deve marcar o evento visualmente no calendário de todos os utilizadores com a etiqueta “CANCELADO” ou “ALTERADO”. 3. O sistema deve gerar um alerta visual na página inicial do jogador para mudanças ocorridas nas últimas 24 horas.
Relevância para a aplicação	Evita que jogadores apareçam no campo à hora errada ou debaixo de chuva por falta de aviso centralizado.

Tabela 25: RF13 - Gestão de Alterações de Última Hora

Requisito de utilizador	Os Proprietários devem poder remover ou editar as suas viaturas e lugares disponíveis.
Fonte	Contexto (“Tudo é gerido no momento... sem uma lista clara de lugares”).
Area do sistema	Logística de Transporte
Requisitos de sistema	1. O sistema deve permitir ao proprietário eliminar uma viatura registada (ex: venda do carro). 2. O sistema deve permitir alterar o número de lugares disponíveis para uma viagem específica, caso o condutor precise de transportar carga ou família. 3. Se a remoção de lugares afetar reservas já feitas, o sistema deve listar os jogadores que ficaram sem lugar para que o Treinador possa intervir.
Relevância para a aplicação	Dá flexibilidade à logística, garantindo que a lista de lugares disponíveis é real e não uma estimativa fixa.

Tabela 26: RF14 - Gestão de Ativos de Transporte

Requisito de utilizador	O Treinador deve poder registar o resultado final dos jogos oficiais.
Fonte	Contexto (“A ausência de um registo fidedigno... impede qualquer avaliação do rendimento”).
Area do sistema	Operações de Jogo
Requisitos de sistema	1. O sistema deve disponibilizar campos numéricos para “Golos Marcados” e “Golos Sofridos” após a hora de fim do jogo. 2. O sistema deve permitir adicionar uma breve “Crónica do Jogo” para memória futura. 3. O sistema deve atualizar automaticamente o histórico de vitórias/derrotas do clube.
Relevância para a aplicação	Transforma o sistema no verdadeiro “caderno do Quim”, preservando a história desportiva da Vila de São Vicente da Beira.

Tabela 27: RF15 - Registo Histórico de Resultados

Justificação da Revisão

Ao rever os requisitos anteriores à luz destas adições, garanto que:

Segurança: O RF12 ataca o problema mais grave de saúde/segurança relatado.

Flexibilidade: O RF13 e RF14 reconhecem que a vida na Beira é imprevisível (chuva, carros avariados).

Ciclo de Vida: O RF11 fecha o ciclo de vida do utilizador no sistema (Entrada -> Participação -> Saída).

Após analisar o output, a equipa de desenvolvimento entendeu que, com os requisitos adicionados

1.8. Especificação do software (SRS – IEEE 830/29148)

Prosseguindo o processo de para uma especificação dos requisitos de software, para implementar este ponto, propusemos o seguinte prompt ao agente, que produziu o seguinte output.

Prompt

Deves agora redigir formalmente, sobre a norma ISO 29148, a Especificação dos Requisitos de Software, baseando-te no total de 20 requisitos gerados.

Caso haja ambiguidades, deves basear-te no livro Software Engineering, de Ian Sommerville.

Este prompt adveio do requisito de gerar uma descrição formal, nos conformes da norma ISO 29148, proposto pelo IEEE, para seguindo também a obra recomendada da disciplina [1]. Com isso, foi pedido que, após uma eliciação mais informal dos requisitos, estes fossem dispostos e enunciados segundo a norma.

Analisando o output, é perceptível que o agente introduziu ambiguidades ao construir os requisitos específicos, porque ignorou a estrutura que tinha feito, onde eram descritos os requisitos funcionais e não funcionais, e eram compostos por múltiplos requisitos de sistema.

Dada essa falha, propusemos o seguinte prompt:

Prompt

Não está correto. Nos requisitos específicos, deves apenas organizar os requisitos já especificados, copiando apenas.

Output

1.8.1. Introdução

1.8.1.1. Propósito

O objetivo deste documento é definir as especificações funcionais e não funcionais para o sistema de gestão do Sport Clube “Os Beirões”. O sistema visa substituir processos manuais e informais (papel, Excel disperso, WhatsApp) por uma solução centralizada que suporte a “carolice” da Vila de São Vicente da Beira de forma organizada.

1.8.1.2. Âmbito (Scope)

Conforme sugere Sommerville, o sistema é um sistema socio-técnico que deve integrar-se na rotina de utilizadores com baixa literacia digital. O foco reside na gestão de ativos (jogadores), operações desportivas (treinos/jogos) e logística comunitária (boleias), garantindo a sobrevivência da mística do clube através da eficiência digital.

1.8.2. Descrição Geral

1.8.2.1. Características dos Utilizadores

Aplicando o método de Sommerville de análise de stakeholders, a interface deve ser minimalista para atender ao Presidente (Sr. Quim), ao Treinador (Mister Zé) e aos Jogadores.

1.8.2.2. Restrições (Constraints)

Literacia Digital: Interface simplificada.

Orçamento: Custo de manutenção nulo (Cloud Free Tier).

1.8.3. Requisitos Específicos (Conforme ISO/IEC/IEEE 29148)

Nesta secção, os requisitos são apresentados conforme eliciados anteriormente, garantindo que são necessários e verificáveis.

1.8.3.1. Requisitos Funcionais (RF)

Requisito de utilizador	O Presidente deve ser capaz de registar novos jogadores no sistema.
Fonte	Sr. Quim (Entrevista - Problema dos dossiers poeirentos)
Área do sistema	Gestão de Plantel
Requisitos de sistema	1. O sistema deve disponibilizar um formulário de inserção de dados. 2. O utilizador deve fornecer: Nome Completo, Data de Nascimento, Posição Preferencial e Contacto Telefónico. 3. O sistema deve validar se o campo “Nome” e “Contacto” estão preenchidos antes de guardar. 4. O sistema deve gerar um identificador único para o jogador após o sucesso da operação.
Relevância para a aplicação	Essencial para centralizar a informação que atualmente está dispersa em papéis e dossiers.

Tabela 28: RF01 - Registo de Jogadores

Requisito de utilizador	O Presidente deve poder consultar e editar os dados dos jogadores inscritos.
Fonte	Sr. Quim (Contexto - Dados de idades e posições desatualizados)
Área do sistema	Gestão de Plantel
Requisitos de sistema	1. O sistema deve listar todos os jogadores por ordem alfabética de nome. 2. O sistema deve permitir a alteração de qualquer campo (exceto ID único) de um jogador existente. 3. O sistema deve guardar a data da última atualização de cada ficha de jogador.
Relevância para a aplicação	Garante que a secretaria tenha dados fidedignos para as fichas de jogo de domingo.

Tabela 29: RF02 - Consulta e Atualização de Jogadores

Requisito de utilizador	O Treinador deve poder criar eventos de treino no calendário.
Fonte	Mister Zé (Entrevista - Incerteza no calendário)
Área do sistema	Planeamento Desportivo
Requisitos de sistema	1. O sistema deve permitir inserir a data e a hora de início do treino. 2. O sistema deve permitir selecionar o local do treino (Campo Principal ou Sintético). 3. O sistema deve impedir a criação de treinos em baterias passadas.
Relevância para a aplicação	Combate a “névoa de incerteza” sobre as datas dos treinos que se perdem no WhatsApp.

Tabela 30: RF03 - Criação de Treinos

Requisito de utilizador	O Treinador deve registar as presenças dos jogadores em cada treino.
Fonte	Mister Zé (Contexto - Ausência de registo fidedigno de assiduidade)
Área do sistema	Controlo de Rendimento
Requisitos de sistema	1. Para cada treino criado, o sistema deve apresentar a lista de todos os jogadores ativos. 2. O sistema deve permitir marcar cada jogador como “Presente” ou “Ausente” através de uma caixa de seleção. 3. O sistema deve permitir adicionar uma nota de texto curta (ex: “Lesionado”) ao registo de presença.
Relevância para a aplicação	Permite ao treinador avaliar de forma justa a assiduidade e o compromisso dos atletas.

Tabela 31: RF04 - Registo de Presenças

Requisito de utilizador	O Treinador deve criar convocatórias para jogos oficiais.
Fonte	Mister Zé (Entrevista - “Exercício de adivinhação”)
Área do sistema	Operações de Jogo
Requisitos de sistema	1. O sistema deve permitir selecionar um Jogo previamente criado. 2. O utilizador deve selecionar os jogadores a convocar a partir da lista geral de inscritos. 3. O sistema deve enviar uma notificação interna aos jogadores selecionados.
Relevância para a aplicação	Formaliza o processo de convocatória, retirando-o do ruído das conversas paralelas.

Tabela 32: RF05 - Criação de Convocatória

Requisito de utilizador	O Jogador deve responder à convocatória (Confirmar/Recusar).
Fonte	Atleta Tiago (Entrevista - Botão de Sim ou Não)
Área do sistema	Operações de Jogo
Requisitos de sistema	1. O sistema deve apresentar ao jogador as opções “Vou” e “Não Vou” para cada convocatória pendente. 2. O sistema deve registar a resposta e a hora em que foi efetuada. 3. O sistema deve permitir que o jogador altere a resposta até 12 horas antes do jogo.
Relevância para a aplicação	Dá ao treinador a certeza de quem estará disponível para o jogo com antecedência.

Tabela 33: RF06 - Resposta à Convocatória

Requisito de utilizador	O Proprietário deve disponibilizar lugares na sua viatura para boleia.
Fonte	Contexto (Caos de última hora nas viagens)
Área do sistema	Logística de Transporte
Requisitos de sistema	1. O utilizador deve selecionar a sua viatura registada e associá-la a um Jogo específico. 2. O utilizador deve indicar o número de lugares disponíveis para transporte (excluindo o condutor). 3. O sistema deve apresentar a lista de lugares como “Vagos”.
Relevância para a aplicação	Ponto central para resolver a desorganização das viagens para jogos fora.

Tabela 34: RF07 - Oferta de Boleia

Requisito de utilizador	O Jogador deve poder reservar um lugar numa viatura disponível.
Fonte	Contexto (Jogadores que ficam “a pé” na praça da vila)
Área do sistema	Logística de Transporte
Requisitos de sistema	1. O sistema deve listar todas as viaturas com lugares vagos para o jogo selecionado. 2. O jogador deve selecionar uma viatura para efetuar a reserva. 3. O sistema deve decrementar o número de lugares vagos da viatura após a reserva. 4. O sistema deve impedir reservas em viaturas com 0 lugares vagos.
Relevância para a aplicação	Garante que todos os jogadores tenham transporte confirmado antes da hora de partida.

Tabela 35: RF08 - Reserva de Lugar em Boleia

Requisito de utilizador	O Dirigente deve publicar comunicados oficiais para todos os membros.
Fonte	Sr. Quim (Entrevista - Informação perdida no WhatsApp)
Área do sistema	Comunicação Centralizada
Requisitos de sistema	1. O sistema deve disponibilizar um campo de título e um corpo de texto para o comunicado. 2. O sistema deve marcar o comunicado com a data de publicação e o nome do autor. 3. O sistema deve permitir anexar um ficheiro PDF (opcional).
Relevância para a aplicação	Cria um canal oficial de comunicação, eliminando o fosso entre direção e atletas.

Tabela 36: RF09 - Publicação de Comunicados

Requisito de utilizador	Todos os membros devem poder ler os comunicados publicados.
Fonte	Sr. Quim (Entrevista - Informação perdida no WhatsApp)
Área do sistema	Comunicação Centralizada
Requisitos de sistema	1. O sistema deve disponibilizar uma lista com paginação com os títulos, data e autor dos últimos 10 comunicados. 2. O sistema deve permitir que os utilizadores acedam às informações completas de cada comunicado.
Relevância para a aplicação	Cria um canal oficial de comunicação, eliminando o fosso entre direção e atletas.

Tabela 37: RF10 - Leitura de Comunicados

Requisito de utilizador	Todos os utilizadores devem autenticar-se para aceder ao sistema.
Fonte	Contexto (Necessidade de centralização e segurança de dados)
Área do sistema	Segurança e Acessos
Requisitos de sistema	1. O sistema deve solicitar o contacto telefónico e uma palavra-passe. 2. O sistema deve validar as credenciais contra a base de dados de utilizadores ativos. 3. O sistema deve redirecionar o utilizador para o seu perfil específico (Treinador, Jogador, Dirigente).
Relevância para a aplicação	Protege os dados sensíveis dos jogadores e garante que as operações são feitas pelos perfis corretos.

Tabela 38: RF11 - Autenticação

Requisito de utilizador	O Presidente deve poder remover utilizadores que já não pertençam ao clube.
Fonte	Derivação Lógica (Limpeza de base de dados).
Área do sistema	Gestão de Utilizadores
Requisitos de sistema	1. O sistema deve permitir ao administrador selecionar um utilizador e marcar a conta como “Inativa”. 2. O sistema deve impedir que utilizadores removidos/inativos iniciem sessão. 3. O sistema deve manter os dados históricos associados ao utilizador para fins estatísticos, mas ocultá-lo das listas ativas.
Relevância para a aplicação	Evita que a base de dados fique desorganizada com membros que já abandonaram o clube.

Tabela 39: RF12 - Remoção de Utilizadores

Requisito de utilizador	O Treinador e o Presidente devem ter acesso imediato ao contacto de urgência de cada jogador.
Fonte	Contexto (“Contacto de urgência do novo guarda-redes está apenas gravado na memória volátil do Mister Zé”).
Área do sistema	Segurança e Saúde
Requisitos de sistema	1. O formulário de registo de jogador deve incluir obrigatoriamente um campo “Contacto de Emergência” (Nome e Telefone). 2. O sistema deve exibir um botão de “Chamada de Emergência” no perfil de cada jogador, visível apenas para a Direção e Treinador. 3. Ao clicar no botão, o telemóvel do utilizador deve iniciar automaticamente a chamada para o contacto registado.
Relevância para a aplicação	Garante que em caso de lesão a informação não dependa da memória de ninguém.

Tabela 40: RF13 - Gestão de Contactos de Emergência

Requisito de utilizador	O Treinador deve poder cancelar ou alterar a hora de eventos (treinos/jogos) de forma rápida.
Fonte	Contexto (“Alteração da hora de um treino devido à chuva... deixa jogadores à deriva”).
Área do sistema	Planeamento Desportivo
Requisitos de sistema	1. O sistema deve permitir editar a data, hora ou estado (Ativo/Cancelado) de qualquer evento criado. 2. Ao alterar ou cancelar um evento, o sistema deve marcar o evento visualmente no calendário com a etiqueta “CANCELADO” ou “ALTERADO”. 3. O sistema deve gerar um alerta visual na página inicial do jogador para mudanças recentes.
Relevância para a aplicação	Evita que jogadores apareçam no campo à hora errada por falta de aviso centralizado.

Tabela 41: RF14 - Gestão de Alterações de Última Hora

Requisito de utilizador	Os Proprietários devem poder remover ou editar as suas viaturas e lugares disponíveis.
Fonte	Contexto (“Tudo é gerido no momento... sem uma lista clara de lugares”).
Área do sistema	Logística de Transporte
Requisitos de sistema	1. O sistema deve permitir ao proprietário eliminar uma viatura registada. 2. O sistema deve permitir alterar o número de lugares disponíveis para uma viagem específica. 3. Se a remoção de lugares afetar reservas feitas, o sistema deve listar os jogadores afetados para intervenção.
Relevância para a aplicação	Garante que a lista de lugares disponíveis é real e flexível a imprevistos.

Tabela 42: RF15 - Gestão de Ativos de Transporte

Requisito de utilizador	O Treinador deve poder registar o resultado final dos jogos oficiais.
Fonte	Contexto (“A ausência de um registo fidedigno... impede qualquer avaliação do rendimento”).
Área do sistema	Operações de Jogo
Requisitos de sistema	1. O sistema deve disponibilizar campos numéricos para “Golos Marcados” e “Golos Sofridos” após o jogo. 2. O sistema deve permitir adicionar uma breve “Crónica do Jogo”. 3. O sistema deve atualizar automaticamente o histórico de vitórias/derrotas.
Relevância para a aplicação	Transforma o sistema no verdadeiro “caderno do Quim”, preservando a história desportiva.

Tabela 43: RF16 - Registo Histórico de Resultados

1.8.3.2. Requisitos Não Funcionais (RNF)

Requisito de utilizador	O sistema deve ser extremamente fácil de usar por pessoas com pouca experiência digital.
Fonte	Contexto (Baixa literacia digital dos utilizadores principais)
Área do sistema	Usabilidade
Requisitos de sistema	1. A interface deve utilizar ícones grandes e texto com contraste elevado. 2. Ações principais não devem exigir mais de 3 toques no ecrã. 3. O sistema deve evitar jargão técnico, usando termos como “Ir ao Jogo”.
Relevância para a aplicação	Crítico para a adoção; se for complexo, os utilizadores voltarão ao papel.

Tabela 44: RNF01 - Simplicidade de Interface

Requisito de utilizador	O sistema deve funcionar em telemóveis antigos e com pouca internet.
Fonte	Contexto (Foco na mobilidade e orçamento inexistente)
Área do sistema	Desempenho
Requisitos de sistema	1. O tamanho total da página inicial não deve exceder 2MB. 2. Carregamento da lista de convocatórias em menos de 3 segundos em redes 3G. 3. Responsividade para ecrãs pequenos (mínimo 320px).
Relevância para a aplicação	Permite consulta em campo ou na praça da vila.

Tabela 45: RNF02 - Otimização Móvel

Requisito de utilizador	Os dados dos jogadores devem estar protegidos.
Fonte	Engenharia de Software (Segurança/Privacidade)
Área do sistema	Segurança de Dados
Requisitos de sistema	1. Palavras-passe armazenadas utilizando algoritmos de cifra seguros. 2. Acesso a números de telefone restrito a Perfis de Direção e Treinador.
Relevância para a aplicação	Garante conformidade ética e proteção de dados pessoais.

Tabela 46: RNF03 - Proteção de Dados

Requisito de utilizador	O custo de manutenção do sistema deve ser nulo ou residual.
Fonte	Contexto (Orçamento inexistente do clube)
Área do sistema	Arquitetura
Requisitos de sistema	1. Alojamento em serviços de “Free Tier”. 2. Utilização de base de dados sem custos para volumes inferiores a 1GB.
Relevância para a aplicação	Inviabiliza o projeto se gerar despesa fixa ao Sr. Quim.

Tabela 47: RNF04 - Eficiência de Custos

Requisito de utilizador	A informação sobre as boleias deve estar sempre atualizada em tempo real.
Fonte	Contexto (Evitar falhas logísticas)
Área do sistema	Confiabilidade
Requisitos de sistema	1. Atualização imediata da contagem global de lugares após reserva. 2. Controlo de concorrência para evitar reservas duplicadas no último lugar.
Relevância para a aplicação	Evita falhas graves no momento da partida para os jogos.

Tabela 48: RNF05 - Integridade Logística

Aprovação

Responsável Técnico: Equipa de Desenvolvimento

Aprovação do Cliente: Sr. Joaquim “Quim” Barrela (Presidente S.C. Os Beirões)

Analisando a segunda iteração do agente, a equipa entendeu que o output reflete os requisitos esperados, mantendo uma concordância com o contexto até agora descrito e com base nas entrevistas.

Optamos então por prosseguir com esta eliciação de requisitos.

1.9. Casos de uso e diagramas UML

Possuindo os requisitos funcionais e não funcionais declarados e aprovados pela equipa de desenvolvimento, em conjunto com o Sr. Joaquim, é necessário especificar os casos de uso. Propusemos, então, o seguinte prompt:

Prompt

Deves agora proceder a uma extração de casos de uso, a partir dos requisitos, onde deves seguir o método proposto pelo Sr. Professor António Nestor Ribeiro, na sua dissertação de doutoramento.

O output gerado pelo agente correspondente a este prompt revelou algumas ambiguidades na especificação dos use cases, (e.g. um use case com gestão de utilizadores, que implica geralmente operações mais atómicas como Adição, Remoção, Edição e Procura, i.e. operações CRUD), aspeto que poderá levar a ser necessário alterar os mesmos. Por esta razão, optamos por propor o seguinte prompt, que visa impor estas restrições/críticas à solução do agente.

Prompt

Não. Existe ambiguidades na especificação dos use cases, porque o use case deve definir o comportamento de um use case e deve ser atómico, i.e. deve especificar apenas um comportamento, com fluxos alternativos e fluxos de exceção, quando existem. Não deve ser especificado um use case gestão, com múltiplos fluxos alternativos para cada operação do CRUD, dado que isso introduz complexidade desnecessária e não segue o modelo proposto pelo Professor Nestor. Refaz a extração e especificação dos use cases para estes pressupostos.

Output

ID: UC1
Nome: Autenticar no Sistema
Atores: Utilizador (Presidente, Treinador, Jogador)
Descrição: Permite ao utilizador entrar no sistema para aceder às funcionalidades do seu perfil.

Pré-condições: O utilizador deve possuir uma conta registada.

Fluxo Normal:

1. O utilizador introduz o contacto telefónico e a palavra-passe.
2. O sistema valida as credenciais.
3. O sistema inicia a sessão e apresenta o ecrã principal correspondente ao perfil.

Fluxos Exceção:

[2a] Credenciais Inválidas

- O sistema informa que as credenciais estão incorretas.

Pós-condições: O utilizador encontra-se autenticado com as permissões do seu cargo.

ID: UC2
Nome: **Registar Novo Jogador**
Atores: Presidente
Descrição: O Presidente adiciona um novo atleta à base de dados oficial do clube.

Pré-condições: O Presidente deve estar autenticado.

Fluxo Normal:

1. O Presidente acede ao formulário de registo de jogador.
2. O Presidente preenche os dados (Nome, Data de Nascimento, Posição, Contacto e Emergência).
3. O Presidente submete o registo.
4. O sistema valida a integridade dos dados e guarda o novo registo.

Fluxos Exceção:

[4a] Dados Obrigatórios em Falta

- O sistema interrompe o registo e destaca os campos vazios.

Pós-condições: O novo jogador consta na lista oficial e pode ser convocado.

ID: UC3
Nome: **Procurar Jogador**
Atores: Treinador, Presidente
Descrição: O Treinador ou Presidente procura jogadores inseridos na base de dados oficial do clube.

Pré-condições: O Utilizador deve estar autenticado

Fluxo Normal:

1. O Utilizador insere o nome do jogador
2. O O sistema encontra e apresenta as informações do utilizador

Fluxos Alternativos

[1a] O Utilizador insere o ID do jogador

- Volta a 2

Fluxos Exceção:

[2a] Jogador não existe na base de dados

- O sistema informa que o jogador não existe.

Pós-condições: Informações do jogador apresentadas.

ID: UC4
Nome: Editar Utilizador
Atores: Utilizador, Presidente
Descrição: Permite a um utilizador alterar os seus próprios dados de registo ou, no caso do Presidente, alterar os dados de qualquer outro utilizador do sistema.

Pré-condições: O utilizador deve estar autenticado.

Fluxo Normal:

1. O utilizador solicita a edição do seu perfil.
2. O sistema apresenta o formulário com os dados atuais (Contacto, Nome, Posição, etc.).
3. O utilizador altera os dados pretendidos.
4. O sistema valida a integridade e guarda as alterações.

Fluxos Alternativos

[1a] Presidente edita perfil de outrem

- O Presidente utiliza a funcionalidade de pesquisa (UC Procurar Jogador).
- O Presidente seleciona a opção “Editar” no perfil do utilizador encontrado.
- O fluxo segue para o passo 2.

Fluxos Exceção:

[4a] Dados Inválidos

- O sistema impede a gravação e sinaliza os campos incorretos.

[1b] Tentativa de Acesso Não Autorizado

- Se um utilizador (não Presidente) tentar aceder ao formulário de edição de outro utilizador, o sistema bloqueia a ação e informa da falta de permissões.

Pós-condições: Os dados do utilizador são atualizados na base de dados central.

ID: UC5
Nome: Efetuar Convocatória
Atores: Treinador
Descrição: O Treinador seleciona os atletas para um jogo específico.

Pré-condições: O jogo deve estar criado no sistema; O Treinador deve estar autenticado.

Fluxo Normal:

1. O Treinador seleciona o jogo pretendido.
2. O sistema apresenta a lista de jogadores ativos.
3. O Treinador seleciona os jogadores.
4. O Treinador confirma a seleção.
5. O sistema notifica os jogadores selecionados.

Fluxos Exceção:

[5a] Nenhum Jogador Selecionado

- O sistema impede a finalização e solicita uma seleção.

Pós-condições: A lista de convocados para o jogo é fixada e as notificações são enviadas.

ID: UC6
Nome: Responder a Convocatória
Atores: Jogador
Descrição: O Jogador informa a sua disponibilidade para um jogo após ser convocado.

Pré-condições: O Jogador deve ter uma convocatória pendente; O Jogador deve estar autenticado.

Fluxo Normal:

1. O Jogador acede ao detalhe da convocatória.
2. O Jogador seleciona a opção “Vou” ou “Não Vou”.
3. O sistema regista a resposta e a data/hora da mesma.

Pós-condições: A disponibilidade do jogador é atualizada no mapa de jogo do Treinador.

ID: UC7
Nome: Disponibilizar Viatura para Boleia
Atores: Proprietário (Jogador ou Dirigente)
Descrição: Um utilizador com viatura oferece lugares para transporte de colegas para um jogo.

Pré-condições: O Jogo deve estar agendado; O Proprietário deve estar autenticado.

Fluxo Normal:

1. O Proprietário seleciona o jogo para o qual deseja oferecer transporte.
2. O Proprietário indica o número de lugares disponíveis (vagas).
3. O sistema publica a oferta de boleia associada ao jogo.

Pós-condições: A oferta de lugares fica visível para reserva por outros jogadores.

ID: UC8
Nome: Reservar Lugar em Viatura
Atores: Jogador
Descrição: Um jogador reserva um dos lugares vagos oferecidos por um colega ou dirigente.

Pré-condições: O Jogador deve estar convocado para o jogo; Devem existir lugares vagos disponíveis.

Fluxo Normal:

1. O Jogador consulta as viaturas com vagas para o jogo.
2. O Jogador seleciona uma viatura com lugares disponíveis.
3. O sistema regista a reserva e decrementa o contador de vagas da viatura.

Fluxos Exceção:

[3a] Vaga Esgotada

- Sistema informa que a viatura está cheia.

Pós-condições: O lugar do jogador está garantido na viatura selecionada.

ID: UC9
Nome: Registar Presenças em Treino
Atores: Treinador
Descrição: O Treinador marca a comparência dos jogadores na sessão de treino realizada.

Pré-condições: O Treino deve ter ocorrido ou estar a decorrer.

Fluxo Normal:

1. O Treinador abre a lista de jogadores para o treino específico.
2. O Treinador marca as presenças e as ausências.
3. O Treinador guarda o registo.

Pós-condições: O histórico de assiduidade do plantel é atualizado para fins estatísticos.

ID: UC10
Nome: Inativar Utilizador
Atores: Presidente
Descrição: O Presidente remove o acesso de um utilizador ao sistema sem apagar o seu histórico.

Pré-condições: O utilizador alvo deve estar ativo no sistema.

Fluxo Normal:

1. O Presidente pesquisa o utilizador na lista geral.
2. O Presidente seleciona a opção 'Inativar'.
3. O sistema solicita confirmação.
4. O Presidente confirma.
5. O sistema impede logins futuros para essa conta.

Pós-condições: O utilizador deixa de ter acesso ao sistema, mantendo-se a integridade dos dados históricos.

ID: UC11
Nome: Criar Evento
Atores: Treinador, Presidente
Descrição: Permite agendar um novo treino ou jogo no calendário do clube.

Pré-condições: O utilizador deve estar autenticado.

Fluxo Normal:

1. O utilizador acede à funcionalidade de agendamento.
2. O utilizador define o tipo de evento (Treino ou Jogo).
3. O utilizador insere a data, hora de início e local.
4. O sistema valida a disponibilidade do horário e local.
5. O sistema guarda o evento e publica-o no calendário.

Fluxos Exceção:

[4a] Conflito de Horário

- O sistema informa que já existe um evento nesse período e solicita alteração.

Pós-condições: O evento fica disponível para consulta e futuras convocatórias.

ID: UC12
Nome: Editar Evento
Atores: Treinador, Presidente
Descrição: Permite alterar os detalhes (hora, data ou local) de um evento já agendado.

Pré-condições: O evento deve existir no sistema; O utilizador deve estar autenticado.

Fluxo Normal:

1. O utilizador seleciona o evento no calendário.
2. O utilizador altera os dados pretendidos.
3. O sistema valida as novas informações.
4. O sistema guarda as alterações.
5. O sistema notifica automaticamente os jogadores associados ao evento sobre a mudança.

Fluxos Exceção:

[3a] Dados Inválidos -O sistema destaca os erros e impede a gravação. **Pós-condições:** Os detalhes do evento são atualizados no calendário e os interessados são avisados.

ID: UC13
Nome: Cancelar Evento
Atores: Treinador, Presidente
Descrição: Permite anular um treino ou jogo previamente agendado.

Pré-condições: O evento deve estar marcado no sistema.

Fluxo Normal: +O utilizador seleciona o evento a cancelar. +O utilizador confirma a intenção de cancelamento. +O sistema altera o estado do evento para 'Cancelado'. +O sistema notifica todos os jogadores convocados ou previstos para o evento. **Pós-condições:** O evento é removido da lista de atividades ativas.

ID: UC14
Nome: Publicar Comunicado
Atores: Presidente

Descrição: O Presidente envia uma nota oficial ou aviso importante para todos os membros do clube.

Pré-condições: O Presidente deve estar autenticado.

Fluxo Normal:

1. O Presidente redige o título e o corpo do comunicado.
2. O Presidente confirma a publicação.
3. O sistema regista o comunicado com a data e hora atuais.
4. O sistema apresenta o comunicado com destaque na página inicial de todos os utilizadores.

Fluxos Exceção:

[1a] Título Vazio

- O sistema impede a publicação e solicita a inserção de um título.

Pós-condições: A informação oficial fica acessível a todos os utilizadores, reduzindo o ruído do WhatsApp.

ID: UC15
Nome: Ler Comunicado
Atores: Utilizador

Descrição: O Utilizador pretende ler os últimos comunicados publicados no clube.

Pré-condições: O Utilizador deve estar autenticado e existem comunicados.

Fluxo Normal:

1. O Utilizador indica o aviso que pretende ler.
2. O sistema apresenta o comunicado selecionado.

Fluxos Exceção:

[1a] Título Vazio

- O sistema impede a publicação e solicita a inserção de um título.

Pós-condições: Todas as informações ficam apresentadas ao utilizador.

ID: UC16
Nome: **Editar Lugares da Viatura**
Atores: Proprietário
Descrição: Permite ao proprietário ajustar o número de vagas disponíveis na sua viatura para uma deslocação específica.

Pré-condições: A oferta de boleia deve estar ativa; O utilizador deve estar autenticado.

Fluxo Normal:

1. O utilizador acede à sua oferta de boleia no jogo selecionado.
2. O utilizador altera o número total de lugares disponíveis.
3. O sistema valida se a nova capacidade suporta os jogadores já reservados.
4. O sistema atualiza o contador de vagas.

Fluxos Exceção:

[3a] Capacidade Insuficiente

- O sistema informa que o novo número de lugares é inferior ao número de passageiros já confirmados e impede a alteração.

Pós-condições: A disponibilidade de transporte é atualizada para o jogo.

ID: UC17
Nome: **Cancelar Reserva de Boleia**
Atores: Jogador
Descrição: O Jogador desiste do lugar reservado numa viatura para um jogo fora.

Pré-condições: O Jogador deve possuir uma reserva ativa.

Fluxo Normal:

1. O Jogador seleciona a reserva ativa na viatura do colega/dirigente.
2. O sistema valida se o cancelamento ocorre dentro do limite de tempo permitido (até 2 horas antes da partida).
3. O Jogador confirma o cancelamento.
4. O sistema remove a reserva e incrementa uma vaga na viatura.
5. O sistema notifica o Proprietário da viatura sobre a desistência.

Fluxos Exceção:

[2a] Limite de Tempo Excedido

- O sistema informa que já não é possível cancelar a reserva via sistema por estar demasiado próximo da hora de partida, sugerindo contacto direto.

Pós-condições: O lugar fica novamente vago para outros jogadores e a lista de passageiros é atualizada.

ID: UC18
Nome: **Registar Resultado do Jogo**
Atores: Treinador
Descrição: O Treinador introduz o placar final de um jogo terminado.

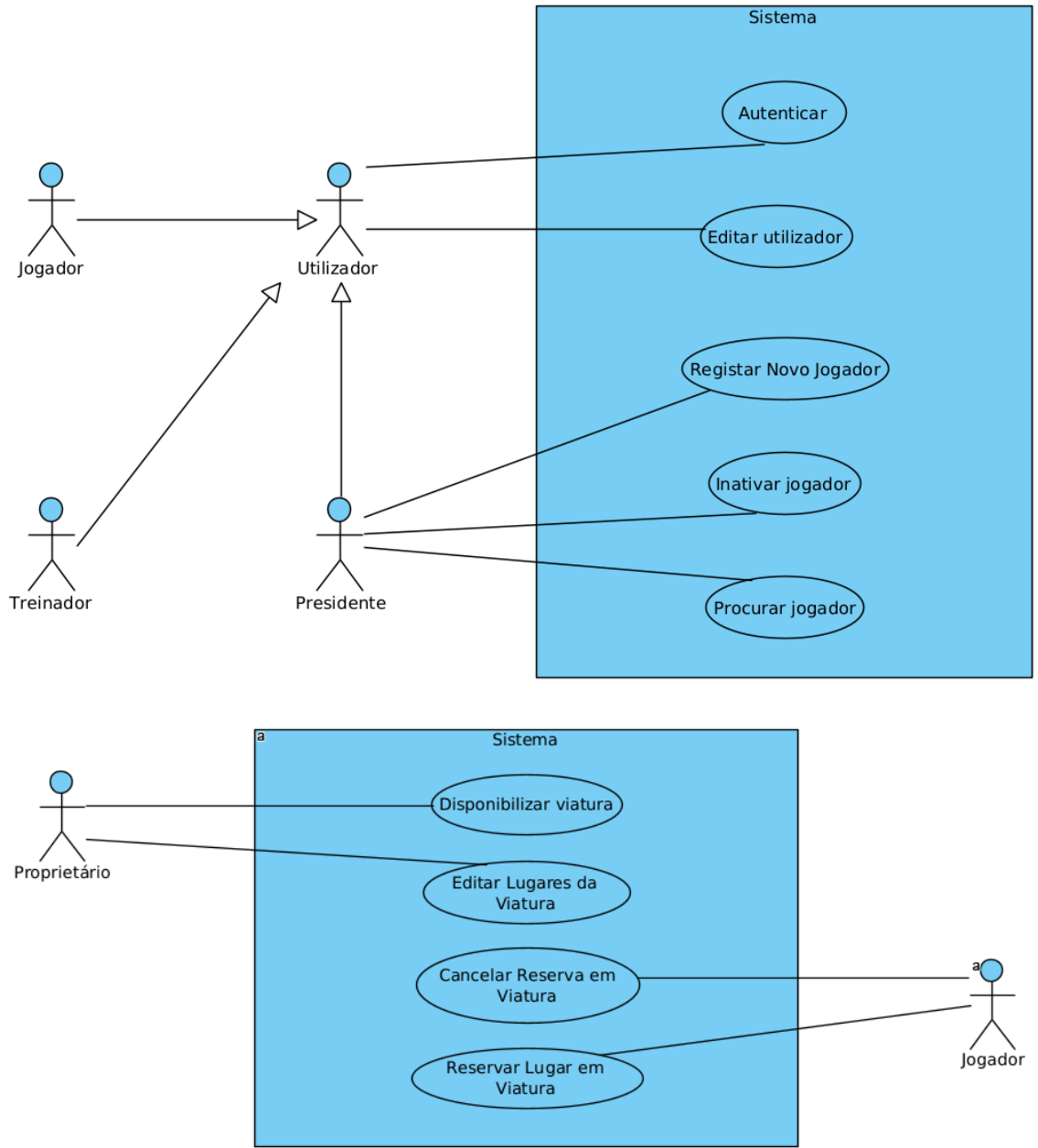
Pré-condições: O jogo deve ter ocorrido.

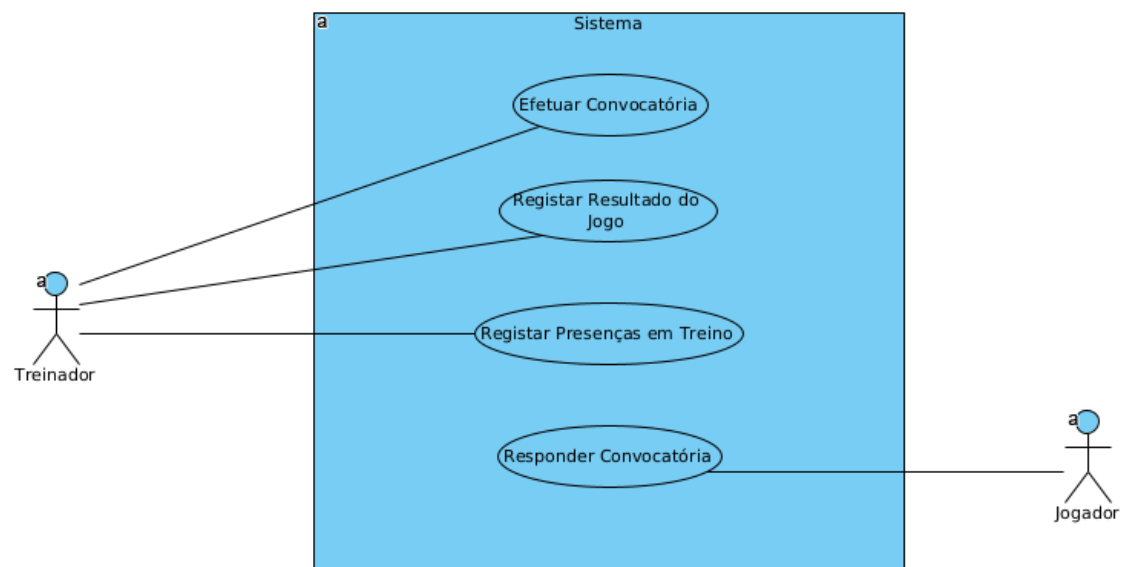
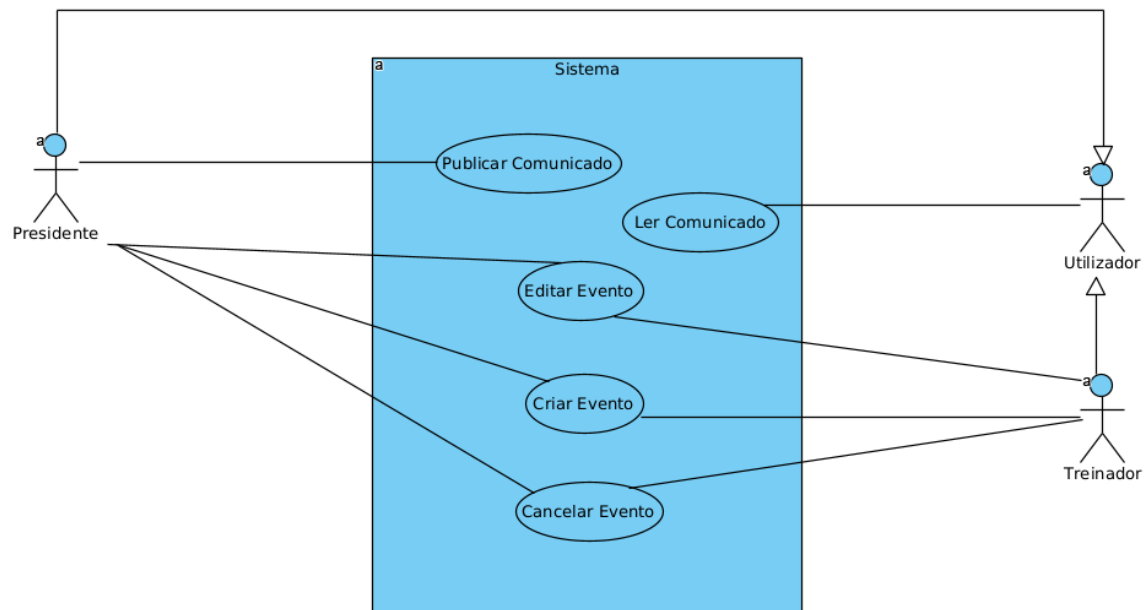
Fluxo Normal:

1. O Treinador seleciona o jogo terminado.
2. O Treinador insere o número de golos marcados e sofridos.
3. O Treinador guarda o resultado.
4. O sistema atualiza o histórico de rendimento da equipa.

Pós-condições: O resultado fica disponível para consulta histórica no “caderno digital”.

Após uma análise do output, entendemos que a especificação dos use cases se encontra completa e concordante com o prévio contexto. Possuem atomicidade suficiente para não ser previsível que seja necessário voltar a alterar esta base do projeto.





2. Arquitetura e Design do Software utilizando LLM

2.1. Definição da arquitetura global do sistema

Com o intuito de obter do agente uma definição da arquitetura global do sistema, propusemos o seguinte prompt:

Prompt

Agora numa fase de modelação do software, deves agora desenvolver uma definição da arquitetura global do sistema, justificando devidamente cada decisão. Deves basear-te no Software Engineering, Ian Sommerville.

Restrições: deves seguir o contexto até agora desenvolvido.

O output que o agente apresentou ficou aquém do esperado, apesar de devidamente justificado, mas, após uma pesquisa, determinou-se a ISO a seguir neste caso, e conclui-se que a proposta não seguiu o ISO 42010, norma do **IEEE** para descrição de arquiteturas de sistemas de software.

Portanto propusemo o seguinte prompt:

Refaz a definição da arquitetura global do sistema, com base no RFC 42010, justificando devidamente cada decisão com base no contexto definido anteriormente.

Deves considerar que pretendemos utilizar a linguagem Python com a framework Flask para um desenvolvimento mais célere.

Deve ser uma aplicação web, para que seja possível executar a aplicação independentemente do dispositivo.

Para o front-end, pretendíamos utilizar Vue.js, dada a familiaridade da equipa de desenvolvimento com a framework, e o CSR ser apropriado para o contexto desta aplicação. A aplicação deve ser assentada numa 3-layered architecture, para que possa existir um desenvolvimento das camadas independentes umas das outra, e possa permitir adaptar as camadas consoante os requisitos do sistema mais facilmente.

Para o backend, pretendemos utilizar um raspberry pi, que deverá possuir uma base de dados PostgreSQL com um tunnel cloudflare, utilizando gunicorn para deploy do servidor flask.

Output

Esta definição de arquitetura para “O Caderno do Quim” segue as diretrizes da norma **ISO/IEC/IEEE 42010:2011**, focando-se na descrição de uma estrutura que responda às preocupações dos *stakeholders* e respeite as restrições de infraestrutura física e lógica solicitadas.

De acordo com **Ian Sommerville**, a arquitetura de um sistema é o esqueleto que permite gerir a complexidade e garantir que requisitos não funcionais, como a manutenibilidade e a portabilidade, sejam atingidos.

2.1.1. Identificação da Arquitetura (ISO 42010)

- **Sistema:** Plataforma de Gestão Integrada S.C. “Os Beirões”.
- **Stakeholders:** Sr. Quim (Presidente), Mister Zé (Treinador), Jogadores, Equipa de Desenvolvimento.
- **Preocupações:** Custo zero de manutenção (infraestrutura caseira), facilidade de acesso via browser e desenvolvimento modular para rápida adaptação.

2.1.2. Estilo Arquitetural: 3-Layered Architecture (Três Camadas)

Seguindo o requisito de desenvolvimento independente e adaptabilidade, o sistema é estruturado em três camadas lógicas distintas. Esta separação permite que a equipa de front-end e back-end trabalhe em paralelo sem bloqueios.

2.1.2.1. Camada de Apresentação (Presentation Layer - Vue.js)

- **Tecnologia:** Vue.js com Client-Side Rendering (CSR).
- **Decisão:** A interface corre no browser do utilizador.
- **Justificação:** O CSR é ideal para o contexto d'Os Beirões, pois, após o carregamento inicial, a navegação entre ecrãs (ex: passar da Convocatória para a Reserva de Boleia) é instantânea. Dada a familiaridade da equipa com Vue.js, garantimos uma UI reativa e intuitiva para utilizadores com baixa literacia digital.

2.1.2.2. Camada de Lógica de Negócio (Business Logic Layer - Flask)

- **Tecnologia:** Python com a framework Flask.
- **Decisão:** Implementação de uma API RESTful.
- **Justificação:** Flask, sendo um *micro-framework*, permite um desenvolvimento célere e leve, essencial para correr num **Raspberry Pi**. Esta camada processa as regras de negócio (ex: validar se ainda é permitido cancelar uma boleia ou se o utilizador tem permissões de Presidente).

2.1.2.3. Camada de Dados (Data Layer - PostgreSQL)

- **Tecnologia:** PostgreSQL com SQLAlchemy (ORM).
- **Decisão:** Base de dados relacional robusta.
- **Justificação:** O PostgreSQL garante a integridade dos dados (essencial para não perder registos de presenças ou exames médicos). O SQLAlchemy atua como ponte, permitindo que a lógica de negócio manipule dados como objetos Python, facilitando a manutenção.

2.1.3. Vista de Infraestrutura e Implementação (Physical View)

Esta secção descreve como o software será alojado de forma a respeitar o orçamento inexistente do clube, utilizando hardware de baixo consumo e custo.

- **Servidor Local (On-premise):** Utilização de um **Raspberry Pi**.
 - **Justificação:** Baixo consumo energético e custo único de aquisição, eliminando mensalidades de servidores Cloud (AWS/Azure).
- **Servidor de Aplicação: Gunicorn (WSGI).**
 - **Justificação:** O servidor de desenvolvimento do Flask não é seguro nem performante. O Gunicorn oferece a robustez necessária para lidar com múltiplos acessos simultâneos (ex: domingo de manhã quando todos verificam as boleias).
- **Exposição e Conetividade: Cloudflare Tunnel.**
 - **Justificação:** Resolve o problema dos ISPs domésticos (IP dinâmico e CGNAT). Permite expor o Raspberry Pi à Web de forma segura sem abrir portas no router da sede ou da casa do Sr. Quim, garantindo que a **Web App** seja acessível em qualquer lugar via HTTPS.

2.1.4. Justificação das Decisões de Design (Contexto vs. Tecnologia)

Decisão	Motivação (Contexto)	Justificação Técnica
Aplicação Web	Diversidade de dispositivos dos jogadores.	Independência de SO (Android/iOS/PC) e zero instalação.
Flask + Python	Necessidade de entrega rápida e código simples.	Celeridade no desenvolvimento e facilidade de manutenção.

Raspberry Pi	Orçamento zero para mensalidades Cloud.	Solução <i>self-hosted</i> durável e de baixo custo operacional.
3-Layered	Desenvolvimento independente e ágil.	Desacoplamento total; facilita a troca da DB ou da UI no futuro.
Cloudflare Tunnel	Falta de IP fixo ou conhecimentos de rede.	Segurança e acessibilidade externa simplificada.

2.1.5. Considerações de Sommerville (Segurança e Disponibilidade)

De acordo com Sommerville, em sistemas baseados em hardware simples (como o Raspberry Pi), a **Fiabilidade** é crítica.

- **Backup:** O PostgreSQL deve ser configurado para backups periódicos para um armazenamento externo (ex: Pen Drive ou Dropbox gratuito), protegendo “O Caderno do Quim” de falhas no cartão SD do Raspberry Pi.
- **Sessões:** Utilização de **JWT (JSON Web Tokens)** para autenticação, permitindo que os jogadores permaneçam logados no telemóvel sem sobrecarregar a memória do servidor local.

Após uma verificação e validação com a equipa, entendemos que este seja uma justificação das decisões adequadas e uma infraestrutura que se adequa ao contexto elaborado.

2.2. Revisão arquitetural

Apesar de haver concordância com as decisões, entendemos que deve ser feita uma revisão para complemento e verificação das decisões tomadas, nomeadamente em relação às políticas de backup dos dados. Isto levou-nos a elaborar o seguinte prompt, seguindo uma abordagem de pedido, seguido de restrições, o que notamos que o agente responde melhor.

Prompt

Deves executar uma revisão arquitetural antes de avançar, questionando todas as decisões, justificando devidamente o que for ambíguo.
Deves especificar a política e esquema processo de backup da base de dados. Para tal, deves considerar o volume de dados, periodicidade, e criticidade, e local.

Output

Esta revisão arquitetural segue os princípios de **Ian Sommerville** sobre a análise de riscos em sistemas críticos e a norma **ISO/IEC/IEEE 42010**, focando na robustez da solução face às limitações do hardware e do contexto social d’Os Beirões.

2.2.1. Revisão Arquitetural: Questionamento e Justificação

Antes de avançarmos para a implementação, é vital colocar a arquitetura “sob stress” para garantir que o sistema não falha quando o Mister Zé mais precisar dele.

2.2.1.1. O Raspberry Pi como Servidor de Produção

- **A Questão:** Um Raspberry Pi utiliza cartões SD que são propensos a corrupção de dados após muitos ciclos de escrita. É seguro confiar nisto para contactos de emergência?
- **Justificação:** A decisão mantém-se pelo **custo zero**, mas com uma mitigação: o sistema deve minimizar escritas em disco (logs em memória) e utilizar uma política de backup agressiva. O PostgreSQL é superior ao SQLite aqui porque lida melhor com quebras de energia súbitas (comum em zonas rurais) através do seu mecanismo de *Write-Ahead Logging* (WAL).

2.2.1.2. Cloudflare Tunnel vs. Port Forwarding

- **A Questão:** Porquê introduzir a dependência de um serviço externo (Cloudflare) em vez de abrir portas no router?
- **Justificação:** Segurança e **Literacia Técnica**. Abrir portas expõe o IP da sede a ataques. O túnel permite que o sistema funcione atrás de CGNAT (comum em ISPs de vilas) sem que o Sr. Quim tenha de configurar regras de firewall complexas. Garante também o HTTPS gratuito, essencial para a confiança dos atletas.

2.2.1.3. Vue.js (CSR) em dispositivos antigos

- **A Questão:** O Client-Side Rendering (CSR) exige mais processamento do telemóvel. Os telemóveis antigos da vila vão aguentar?
- **Justificação:** Sim. A aplicação é funcionalmente simples. O benefício da reatividade do Vue.js (não precisar de recarregar a página para ver se uma vaga de boleia foi preenchida) supera o custo de processamento inicial. O uso de **Gunicorn** no backend libertará o Raspberry Pi para servir apenas os dados (JSON), tornando a resposta célere.

2.2.2. Política e Processo de Backup da Base de Dados

O “Caderno do Quim” digital não pode desaparecer se o Raspberry Pi avariar. Abaixo defino a política de salvaguarda.

2.2.2.1. Análise de Requisitos de Backup

Parâmetro	Especificação	Justificação
Volume de Dados	Muito Baixo (< 50MB/ano)	O clube tem poucas dezenas de atletas e registos de texto.
Periodicidade	Diária (Automática)	O ritmo de alteração é baixo durante a semana, mas crítico no fim de semana.
Criticidade	Muito Alta	A perda de dados significa a paragem total da logística do clube.
Local de Destino	Híbrido (Local + Nuvem)	Proteção contra falha do hardware e perda de conectividade.

2.2.2.2. Esquema do Processo de Backup

O processo será automatizado através de um *cron job* no sistema operativo (Linux) do Raspberry Pi:

1. Backup Local (Snapshot):

- Todos os dias às 03:00 AM, o sistema executa um `pg_dump` da base de dados PostgreSQL.
- O ficheiro é comprimido (`.tar.gz`) e guardado numa partição separada do cartão SD ou, idealmente, numa Pen Drive USB ligada ao Pi (para evitar o desgaste do SD).

2. Backup Remoto (Off-site):

- Após o dump local, o sistema utiliza o **Rclone** ou um script Python para enviar o ficheiro para um serviço de armazenamento gratuito (ex: Google Drive associado ao email do clube ou Dropbox).
- **Justificação:** Garante que, se o Raspberry Pi “arder” ou for roubado da sede, o Sr. Quim pode recuperar tudo num novo dispositivo em minutos.

3. Política de Retenção:

- **Diários:** Manter os últimos 7 dias.
- **Mensais:** Manter o último backup de cada mês durante a época desportiva.
- **Justificação:** Permite recuperar dados apagados acidentalmente pelo Mister Zé sem ocupar espaço excessivo no armazenamento gratuito.

2.2.2.3. Recuperação (Disaster Recovery)

Em caso de falha total do cartão SD:

1. Substituir o cartão SD com uma imagem base (OS + Flask + Postgres).
2. Restaurar o último backup remoto via comando `pg_restore`.
3. O tempo estimado de recuperação (RTO) deve ser inferior a 2 horas.

2.2.3. Conclusão da Revisão

A arquitetura é **coesa**. O risco do hardware (Raspberry Pi) é mitigado pela robustez do PostgreSQL e pela política de backup externa. O uso de **Gunicorn** como servidor WSGI garante que a aplicação Flask não “congele” quando vários jogadores acederem ao mesmo tempo antes de um jogo.

Acreditamos que a revisão estrutural está realizada, dado que foram justificadas algumas ambiguidades deixadas pela arquitetura anterior, nomeadamente em relação às políticas de backup aplicadas à base de dados.

2.3. Diagramas de classes

Com o intuito de desenvolver diagramas de classes para suportar ou representar a arquitetura do sistema, e dado que pretendemos adotar o método proposto pelo Sr. Professor António Nestor Ribeiro. Isto porque o método, por adotar o padrão arquitetural Facade e utilizar interfaces, permite um desacoplamento da lógica de negócio da camada de apresentação, acordando na API o comportamento exposto. Esta lógica permite o design do sistema seguindo um método conhecido pela equipa de desenvolvimento e ainda uma tradução para um sistema Web não alterando o que está descrito nos diagramas. Por estas razões, propusemos o seguinte prompt:

Prompt

Deves agora proceder ao diagrama de classes correspondente às decisões tomadas até ao momento. Deves apresentar o plano em texto para plantUML.

- Deves criar subsistemas, que representem os módulos do sistema.
- Cada subsistema deve possuir um Facade, de forma a que seja possível mapear facilmente a API a ser disponibilizada via Web.
- Deve por fim existir um Facade geral que seja uma junção de todos os Facades dos subsistemas.

Analisando o output, percebemos que o agente, implementou apenas um diagrama de package, onde refere apenas esboços dos diagramas pretendidos. A equipa não aprovou o output, pelo que propusemos o seguinte prompt:

Prompt

A solução apresenta um diagrama de package, com apenas um esboço das API, Facade e classes de cada subsistema. Deves implementar todas as classes necessárias para cada subsistema, sendo possível utilizar uma parte das classes propostas na solução anterior

A solução proposta a este prompt demonstrou que ainda existia falta de informação e incoerência de atributos, métodos e relações de classes com descrição, algo que não pretendemos, tendo a equipa acabado por recusar a solução do agente. Entendemos que o pedido da arquitetura no seu todo é uma tarefa para a qual o agente sacrifica informação, pelo que iremos propor prompts para cada subsistema.

Prompt

Incompleto. Deves ignorar a tua solução e recomeçar. Para simplificar, deves primeiro propor um diagrama de classes para o subsistema de utilizadores, onde deves propor uma interface de um Facade, e o próprio facade que implement os métodos:

- Autenticar
- Editar utilizador
- Registrar Jogador
- Inativar Jogador
- Procurar Jogador

Deves explicitar as classes com os todos seus atributos e métodos. Para relações, não apresentes descrição, apenas multiplicidade e qualificadores, caso se trate de uma relação com comportamento semelhante a um Map ou Dicionário.

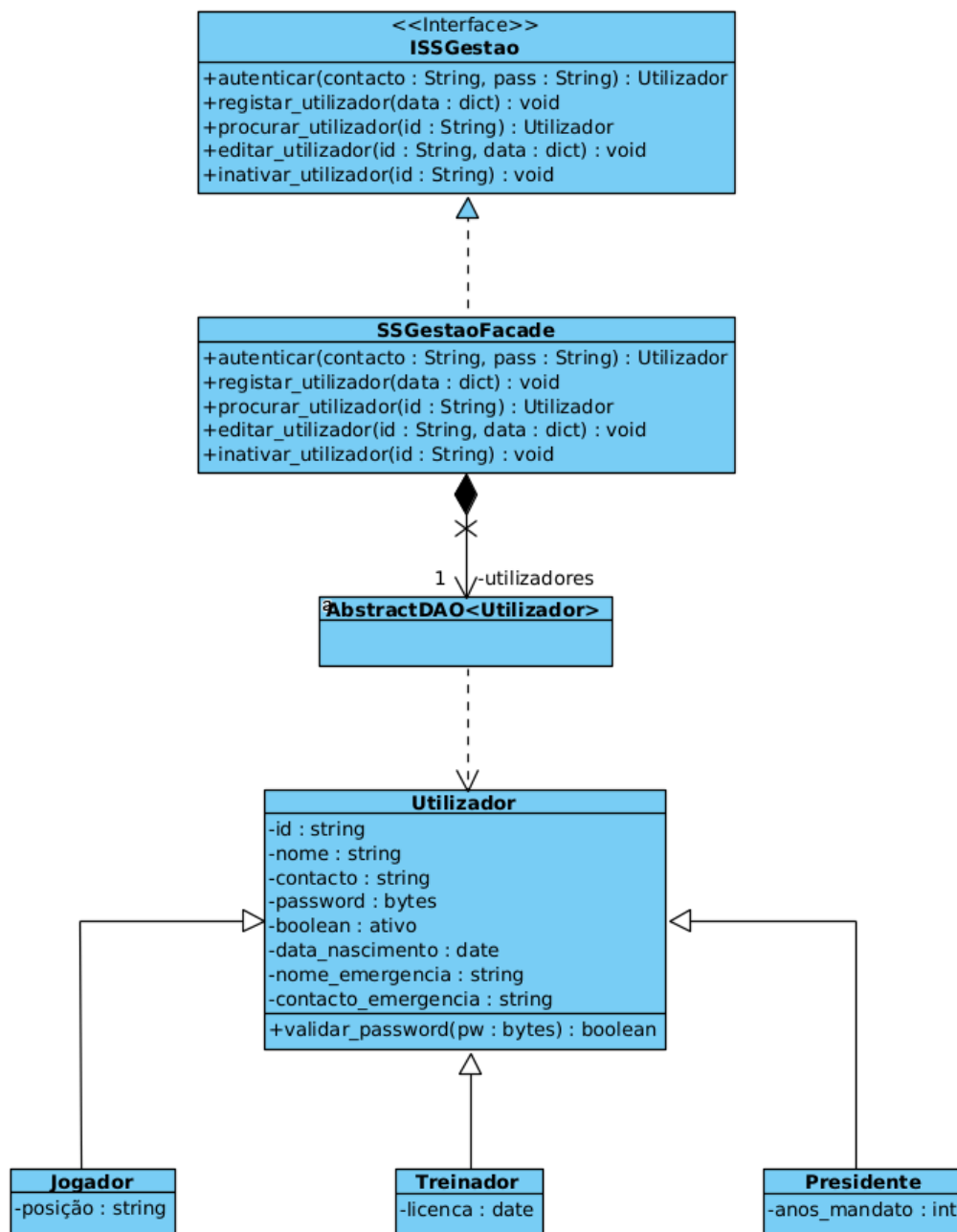


Figura 6: Diagrama de classes para o subsistema de utilizadores

Analisando o output entendemos que o diagrama integra as entidades pertencentes ao problema, apresentando uma solução estrutural para o subsistema. Utiliza um método de abstração de DAO para uma implementação mais célere, dado que ao implementar a API de dicionários ou Mapas, permite desenvolver a lógica de negócio de forma independente da camada de persistência.

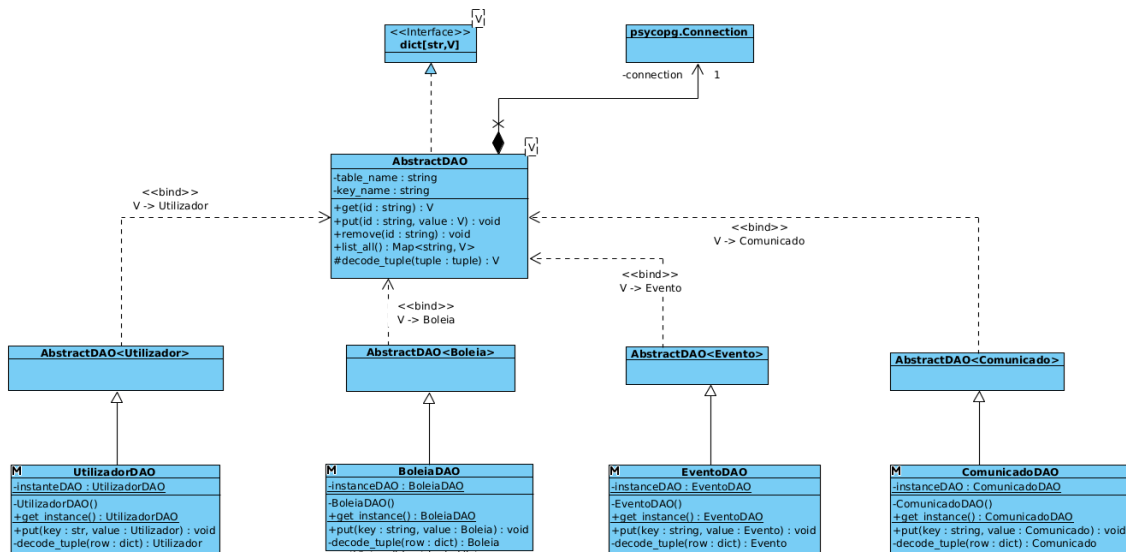


Figura 7: Diagrama de classes para abstração de classes DAO de objetos

Prompt

Deves agora implementar, de forma semelhante, o subsistema de Eventos ou Calendario. Considera que deves possuir o mínimo de DAO possível, e um DAO não necessariamente acede a apenas uma tabela.

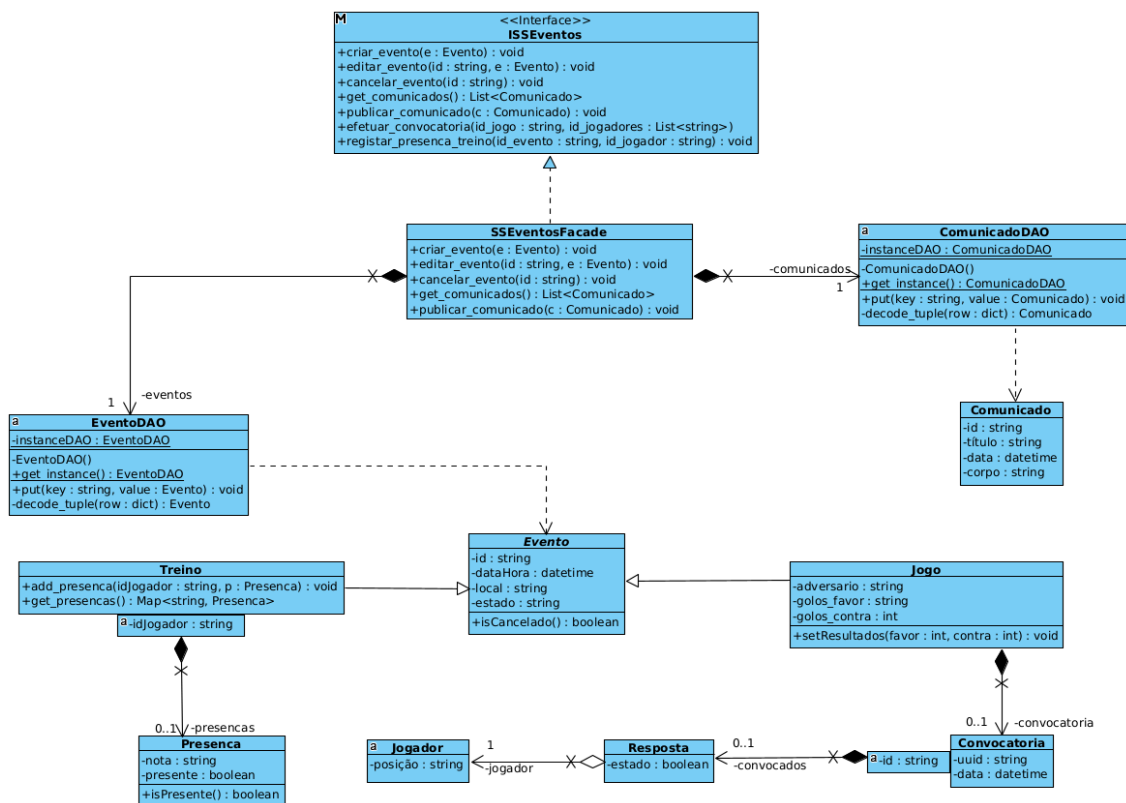
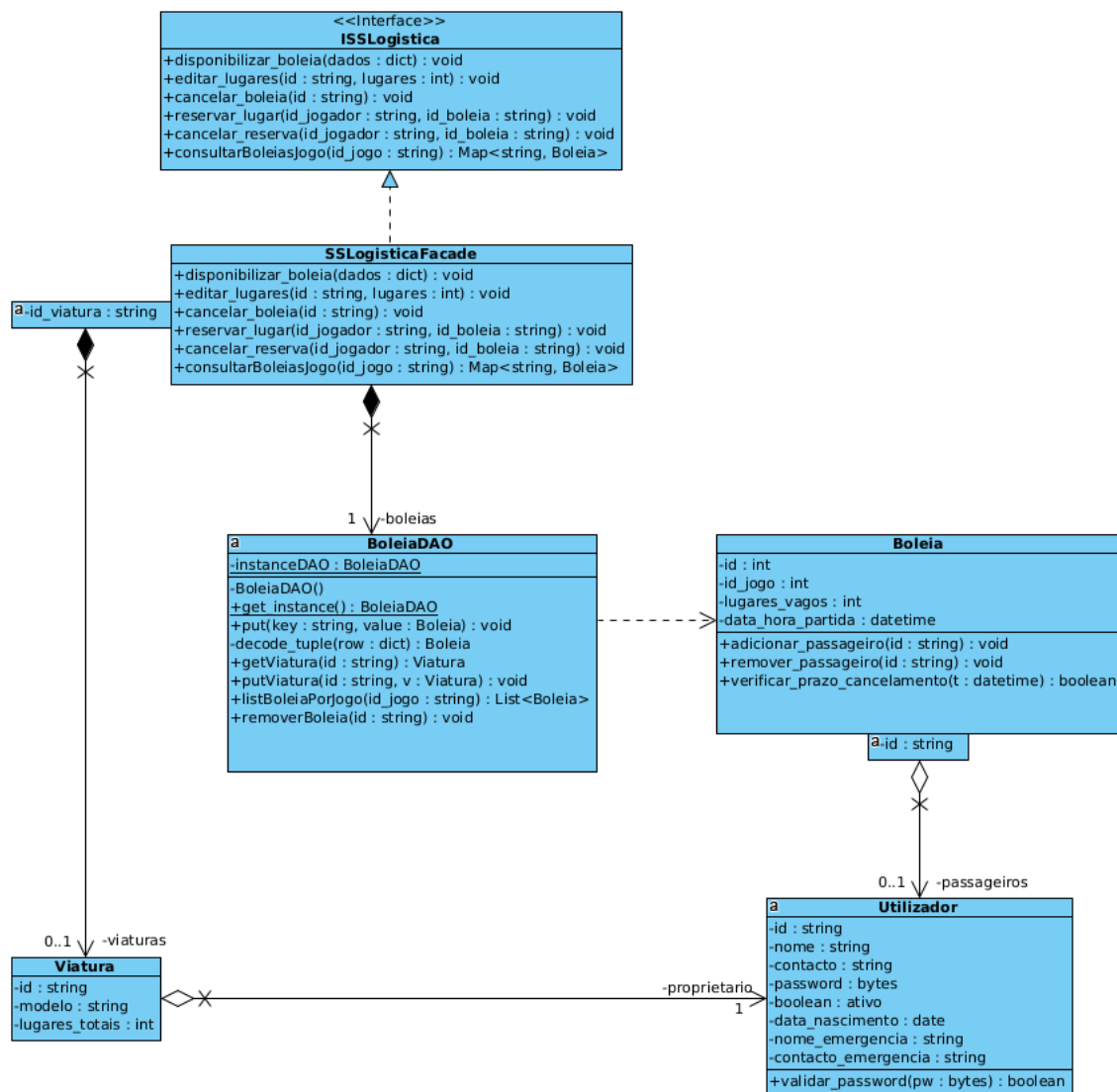


Figura 8: Diagrama de classes para subsistema de eventos

Prompt

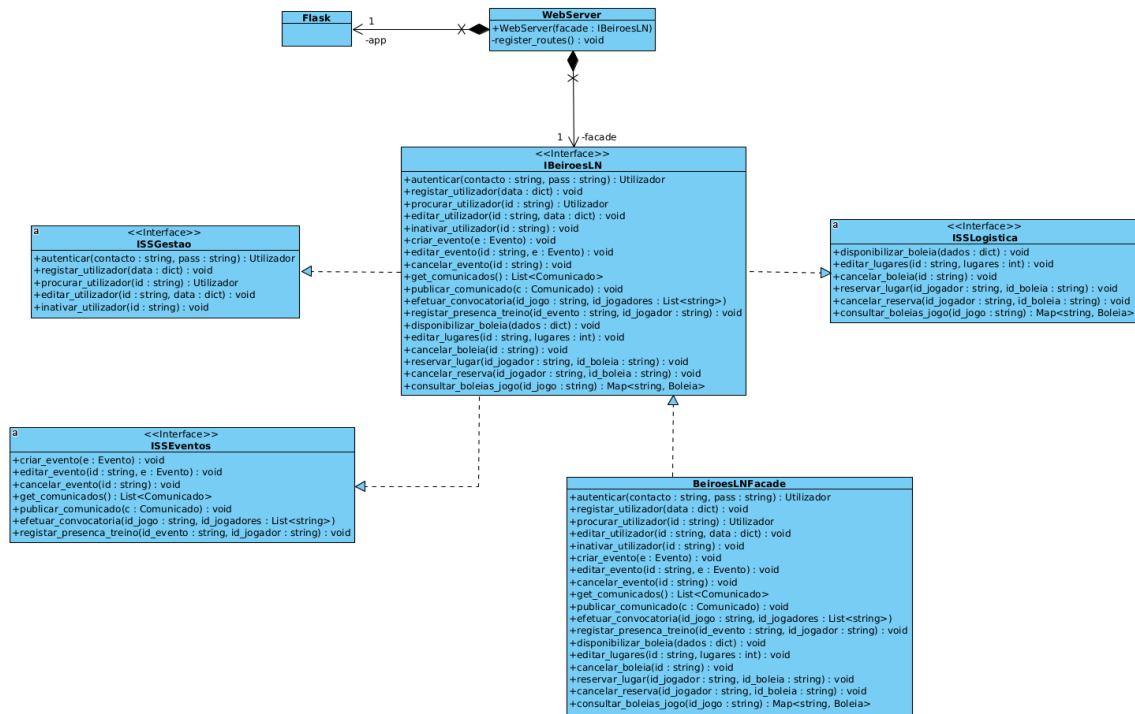
Deves agora implementar o subsistema de boleias ou car pooling, seguindo a mesma forma. Deve existir um BoleiaDAO, que encapsule o acesso à base de dados e que implemente métodos para tratar boleias e viaturas. Como o volume das viaturas não deverá exceder as 25 viaturas, deve existir um Map ou dicionário das viaturas que é carregado para memória quando é iniciado o BoleiaDAO.



Deves agora implementar o subsistema de operações desportivas. Deves seguir a mesma forma ou padrão arquitetural, considerando que pode ser possível apenas carregar para memória informações que não sejam alteradas muitas vezes e que tenham um volume de dados baixo. Deves utilizar relações de agregação em detrimento de utilizar apenas os ids.

Analisando o diagrama de output, o diagrama apresenta um Facade de Operações, algo que não será correto segundo o nosso método, porque o DAO deve estar associado a uma classe do sistema que seja mantida em persistência. Dado que não existe uma classe Operação, a equipa entendeu que não devemos aceitar a solução proposta, pelo que propusemos o seguinte prompt

Prompt



2.4. Diagramas de sequência e componentes necessários

Com o intuito de representar o comportamento dinâmico do sistema e explicitar a colaboração entre a camada de apresentação, a API Web, as fachadas, os objetos de domínio e os mecanismos de persistência, procedemos à geração de um conjunto de diagramas de sequência assistidos por LLM. Para manter coerência com a arquitetura de 3 camadas anteriormente definida, todos os diagramas seguem a mesma estrutura base: o ator interage com a aplicação **Vue.js**, que comunica com a **API Flask**; esta delega o comportamento para a **Facade** do subsistema respetivo, que por sua vez recorre aos **DAO** e objetos de domínio necessários para persistir ou recuperar informação do **PostgreSQL**.

Os diagramas produzidos focam-se nos fluxos dinâmicos essenciais do sistema, cobrindo os subsistemas de gestão de utilizadores, eventos e comunicação, operações desportivas e logística de transportes. Optou-se por não repetir diagramas quase idênticos para operações puramente consultivas ou CRUD triviais, uma vez que o seu comportamento pode ser inferido diretamente a partir dos cenários principais aqui documentados.

2.4.1. Diagrama de sequência para Autenticar no Sistema

Prompt

Deves produzir um diagrama de sequência para o caso de uso "Autenticar no Sistema". Deves seguir as restrições seguintes:

- Segue a arquitetura de 3 camadas já definida para "O Caderno do Quim".
- O front-end é uma aplicação Vue.js e o backend é uma API Flask.
- A API deve comunicar com a SSGestaoFacade, que por sua vez utiliza o UtilizadorDAO.
- Deves representar o ator, a interface, a API, a fachada, o DAO e a base de dados PostgreSQL.
- Inclui um ramo alternativo para credenciais inválidas ou conta inativa.

O output correspondente encontra-se em `diagramasseq/01_autenticar_no_sistema.mmd`. Após exportação para imagem, recomenda-se a seguinte inserção:

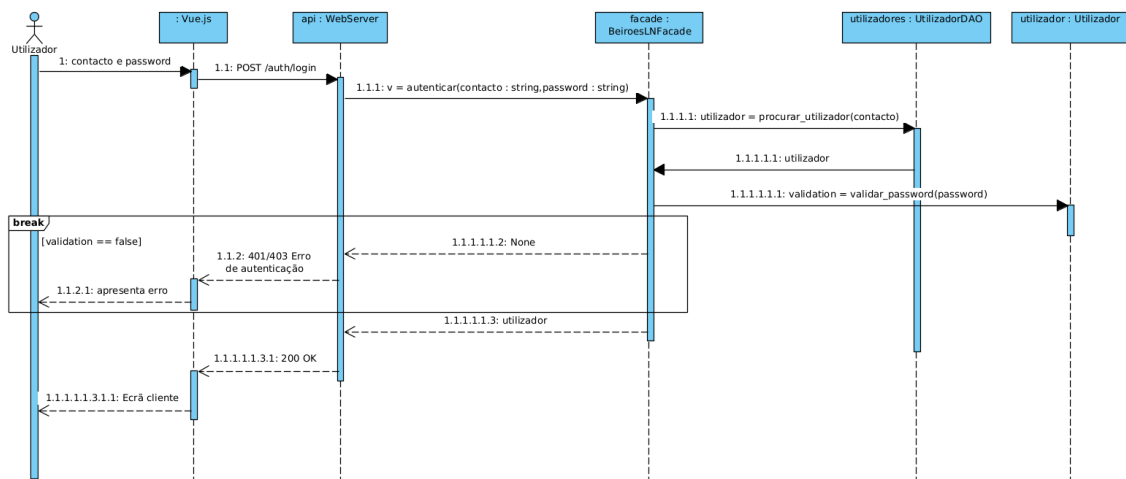


Figura 11: Diagrama de sequencia para autenticar no sistema

O diagrama evidencia a validacao das credenciais na camada de gestao, bem como o retorno do estado de autenticacao para a interface, incluindo o fluxo alternativo para falha de login.

2.4.2. Diagrama de sequencia para Registrar Novo Jogador

Prompt

Deves produzir um diagrama de sequencia para o caso de uso "Registrar Novo Jogador".

Deves seguir as restricoes seguintes:

- O ator principal e o Presidente.
- O fluxo deve passar por Vue.js, API Flask, SSGestaoFacade, UtilizadorDAO e PostgreSQL.
- Deves considerar a criacao de uma entidade Jogador com os dados base e contacto de emergencia.
- Inclui ramos alternativos para dados obrigatorios em falta e duplicacao de registo.

O output correspondente encontra-se em diagramasseq/02_registar_novo_jogador.mmd. Apos exportacao para imagem, recomenda-se a seguinte insercao:

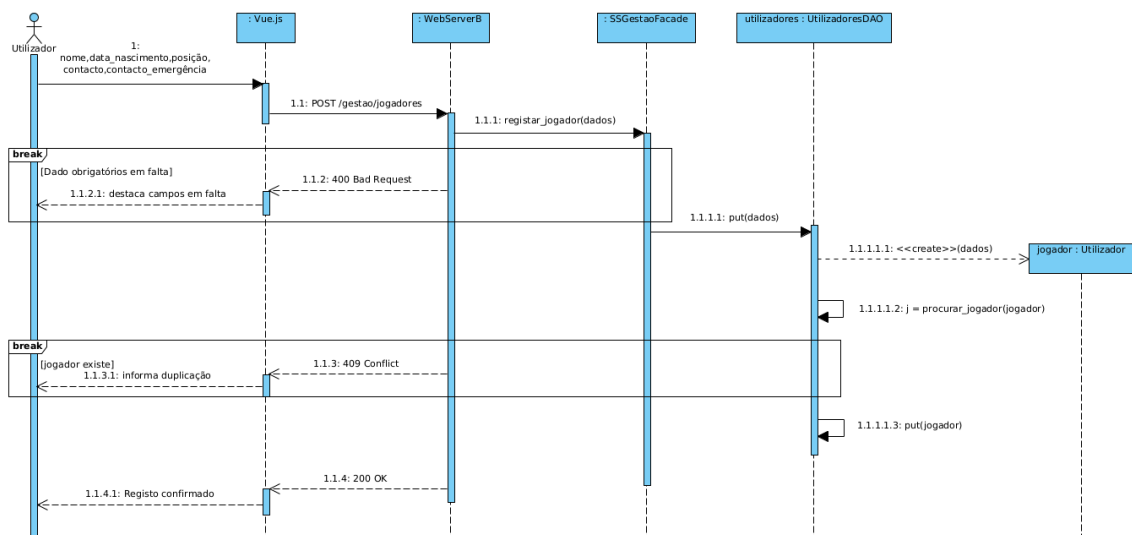


Figura 12: Diagrama de sequencia para registar novo jogador

Este diagrama mostra a validacao dos dados fornecidos pelo Presidente, a verificacao de duplicados e a persistencia do novo jogador, incluindo o tratamento dos principais fluxos de execucao.

2.4.3. Diagrama de sequencia para Criar Evento

Prompt

Deves produzir um diagrama de sequencia para o caso de uso "Criar Evento".

Deves seguir as restricoes seguintes:

- O ator principal pode ser Treinador ou Presidente; usa Treinador para o cenario principal.
- O fluxo deve passar por Vue.js, API Flask, SSEventosFacade, EventoDAO e PostgreSQL.
- O diagrama deve mostrar a validacao de conflitos de horario/local antes da persistencia.
- Deves representar a criacao de um Evento, que pode ser Treino ou Jogo.
- Inclui um ramo alternativo para conflito de agenda.

O output correspondente encontra-se em diagramasseq/03_criar_evento.mmd. Apos exportacao para imagem, recomenda-se a seguinte insercao:

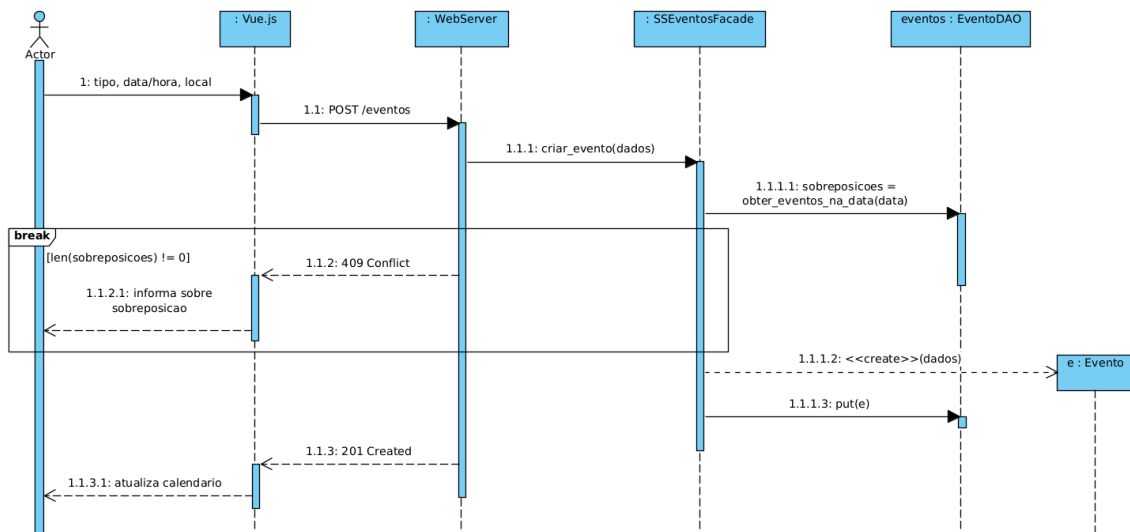


Figura 13: Diagrama de sequencia para criar evento

O diagrama descreve o processo de criacao de um evento no calendario, destacando a validacao previa de conflitos na agenda antes de efetuar a persistencia.

2.4.4. Diagrama de sequencia para Publicar Comunicado

Prompt

Deves produzir um diagrama de sequencia para o caso de uso "Publicar Comunicado".

Deves seguir as restricoes seguintes:

- O ator principal e o Presidente.
- O fluxo deve passar por Vue.js, API Flask, SSEventosFacade, ComunicadosDAO e PostgreSQL.
- Inclui um ramo alternativo para titulo vazio.

O output correspondente encontra-se em diagramasseq/04_publicar_comunicado.mmd. Apos exportacao para imagem, recomenda-se a seguinte insercao:

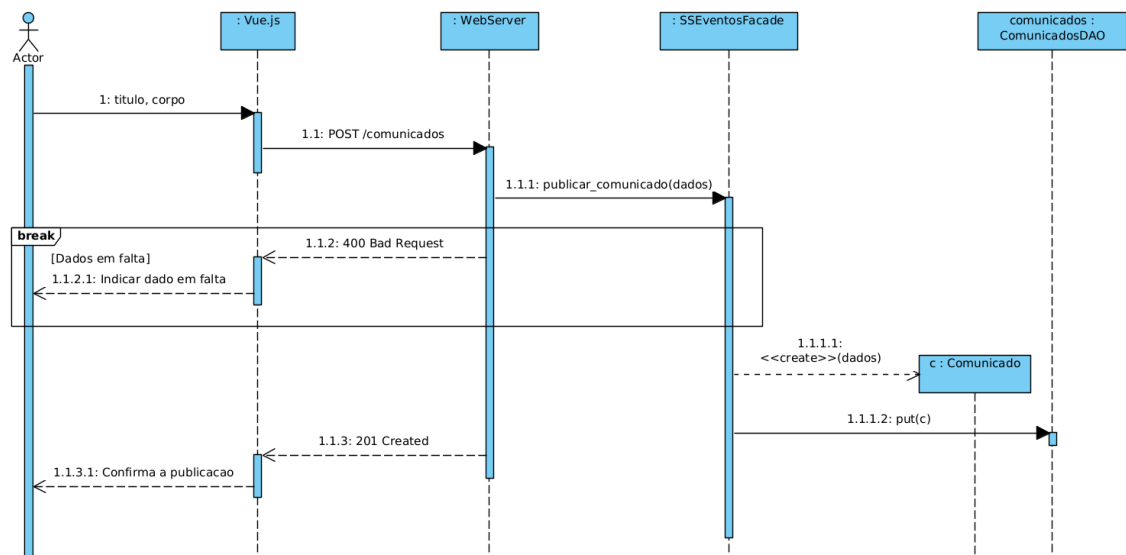


Figura 14: Diagrama de sequencia para publicar comunicado

Este diagrama representa o fluxo dinâmico da criação de comunicados oficiais, reforçando a existência de um canal de comunicação formal alternativo aos grupos de WhatsApp.

2.4.5. Diagrama de sequencia para Registrar Presenças em Treino

Prompt

Deves produzir um diagrama de sequencia para o caso de uso "Registrar Presenças em Treino".

Deves seguir as restricoes seguintes:

- O ator principal e o Treinador.
- O fluxo deve passar por Vue.js, API Flask, SSEventsFacade, EventoDAO, Treino e PostgreSQL.
- Deves mostrar a atualizacao do mapa de presenças de um Treino.
- Inclui um ramo alternativo para treino inexistente ou cancelado.

O output correspondente encontra-se em diagramasseq/05_registar_presencas_treino.mmd. Após exportacao para imagem, recomenda-se a seguinte insercao:

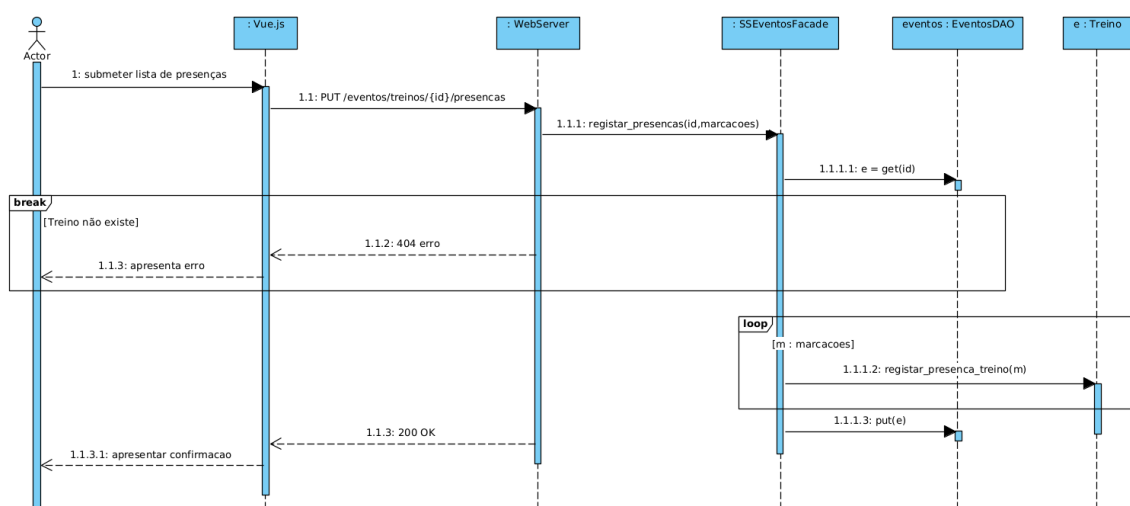


Figura 15: Diagrama de sequencia para registrar presenças em treino

O diagrama torna explicita a forma como o sistema atualiza o conjunto de presenças associado a um treino concreto, preservando a coerencia com o modelo de dominio e com o EventoDAO.

2.4.6. Diagrama de sequencia para Efetuar Convocatoria

Prompt

Deves produzir um diagrama de sequencia para o caso de uso "Efetuar Convocatoria".

Deves seguir as restricoes seguintes:

- O ator principal e o Treinador.
- O fluxo deve passar por Vue.js, API Flask, SSOperacoesFacade, EventoDAO, Jogo, Convocatoria e PostgreSQL.
- Deves mostrar a agregacao da Convocatoria ao Jogo e a adicao de convocados.
- Inclui ramos alternativos para lista vazia e jogo invalido.

O output correspondente encontra-se em diagramasseq/06_efetuar_convocatoria.mmd. Apos exportacao para imagem, recomenda-se a seguinte insercao:

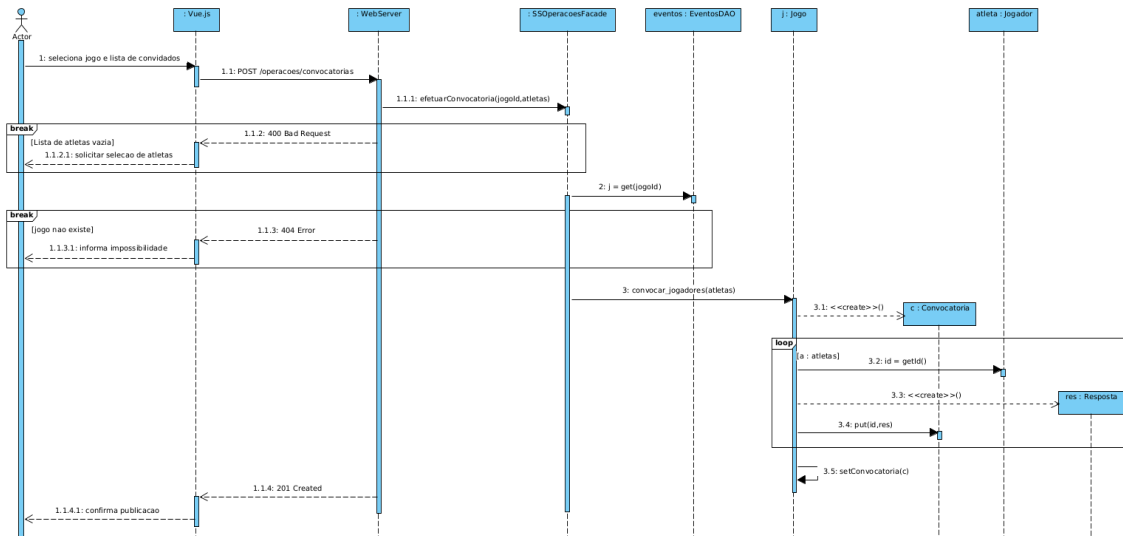


Figura 16: Diagrama de sequencia para efetuar convocatoria

Este diagrama detalha o comportamento nuclear do subsistema de operacoes desportivas, evidenciando a criacao da convocatoria e a sua associacao a um jogo persistido no EventoDAO.

2.4.7. Diagrama de sequencia para Responder a Convocatoria

Prompt

Deves produzir um diagrama de sequencia para o caso de uso "Responder a Convocatoria".

Deves seguir as restricoes seguintes:

- O ator principal e o Jogador.
- O fluxo deve passar por Vue.js, API Flask, SSOperacoesFacade, EventoDAO, Jogo, Convocatoria, Resposta e PostgreSQL.
- Deves mostrar o registo da resposta no mapa de respostas da Convocatoria.
- Inclui um ramo alternativo para jogo sem convocatoria ativa.

O output correspondente encontra-se em diagramasseq/07_responder_convocatoria.mmd. Apos exportacao para imagem, recomenda-se a seguinte insercao:

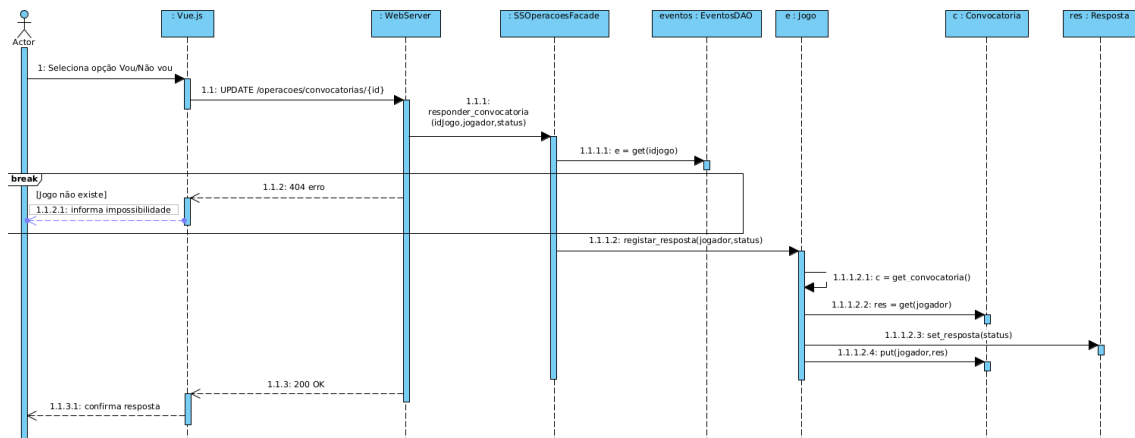


Figura 17: Diagrama de sequencia para responder a convocatoria

O diagrama descreve a atualizacao da disponibilidade do jogador atraves do registo de uma Resposta, refletindo a agregacao existente entre Convocatoria, Resposta e Jogador.

2.4.8. Diagrama de sequencia para Registrar Resultado do Jogo

Prompt

Deves produzir um diagrama de sequencia para o caso de uso "Registrar Resultado do Jogo".

Deves seguir as restricoes seguintes:

- O ator principal e o Treinador.
- O fluxo deve passar por Vue.js, API Flask, SSOperacoesFacade, EventoDAO, Jogo e PostgreSQL.
- Deves mostrar a atualizacao do resultado do Jogo e a sua persistencia.
- Inclui um ramo alternativo para jogo inexistente.

O output correspondente encontra-se em diagramasseq/08_registar_resultado_jogo.mmd. Apos exportacao para imagem, recomenda-se a seguinte insercao:

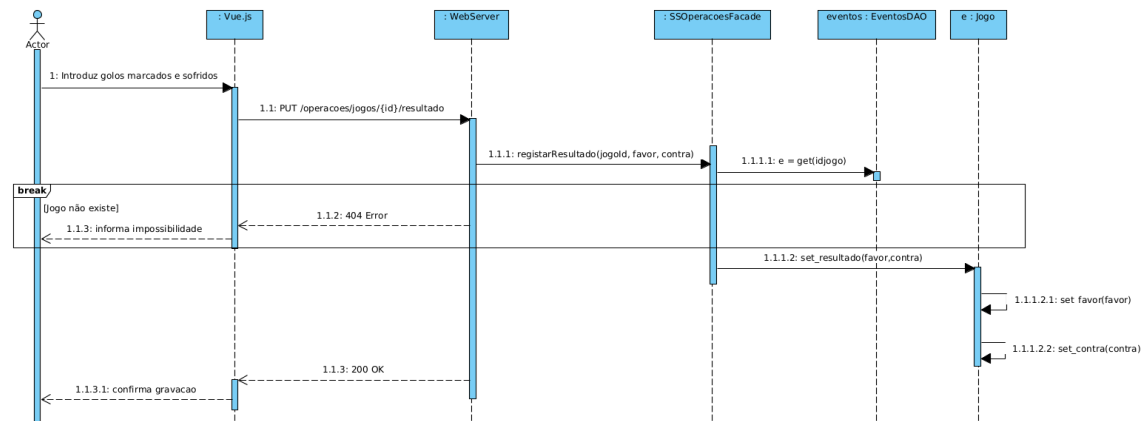


Figura 18: Diagrama de sequencia para registrar resultado do jogo

Este diagrama representa o fecho do ciclo desportivo de um jogo, mostrando a alteracao do estado do objeto Jogo e a respetiva persistencia no sistema.

2.4.9. Diagrama de sequencia para Disponibilizar Viatura para Boleia

Prompt

Deves produzir um diagrama de sequencia para o caso de uso "Disponibilizar Viatura para Boleia".

Deves seguir as restricoes seguintes:

- O ator principal e o Proprietario.

- O fluxo deve passar por Vue.js, API Flask, SSLogisticaFacade, LogisticaDAO, Viatura, Boleia e PostgreSQL.
- Deves mostrar a validacao da viatura e a criacao da Boleia.
- Inclui um ramo alternativo para viatura inexistente ou sem permissao.

O output correspondente encontra-se em diagramasseq/09_disponibilizar_boleia.mmd. Apos exportacao para imagem, recomenda-se a seguinte insercao:

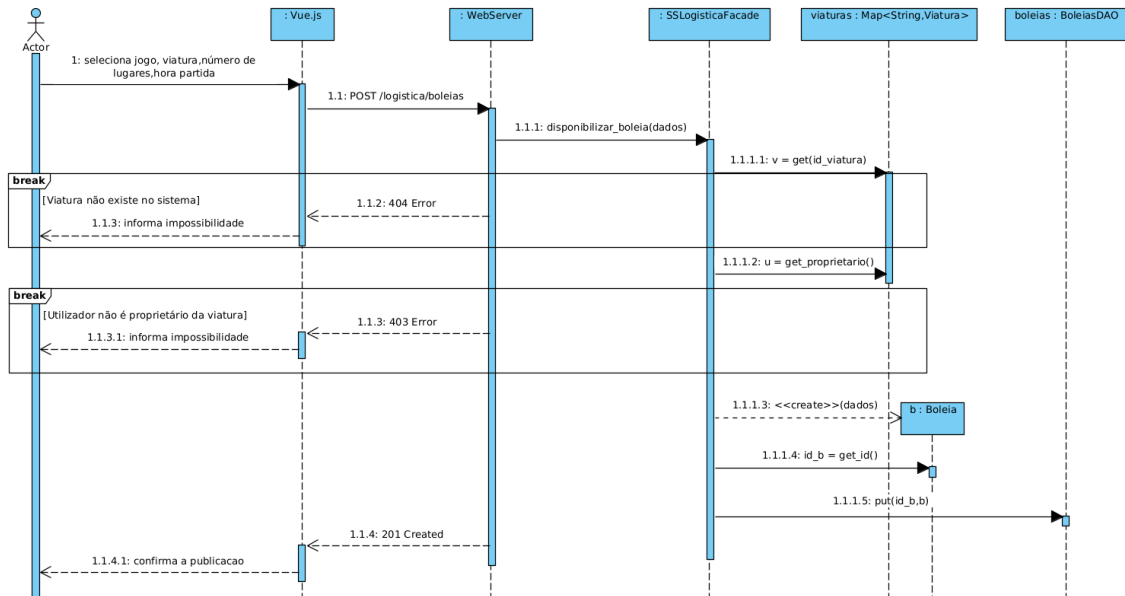


Figura 19: Diagrama de sequencia para disponibilizar viatura para boleia

O diagrama mostra a criacao de uma nova Boleia a partir de uma Viatura validada, mantendo a separacao entre logica de dominio e persistencia concentrada no LogisticaDAO.

2.4.10. Diagrama de sequencia para Reservar Lugar em Viatura

Prompt

Deves produzir um diagrama de sequencia para o caso de uso "Reservar Lugar em Viatura". Deves seguir as restricoes seguintes:

- O ator principal e o Jogador.
- O fluxo deve passar por Vue.js, API Flask, SSLogisticaFacade, LogisticaDAO, Boleia e PostgreSQL.
- Deves mostrar a reserva de um passageiro e a atualizacao atomica dos lugares vagos.
- Inclui ramos alternativos para boleia inexistente e viatura cheia.

O output correspondente encontra-se em diagramasseq/10_reservar_lugar_viatura.mmd. Apos exportacao para imagem, recomenda-se a seguinte insercao:

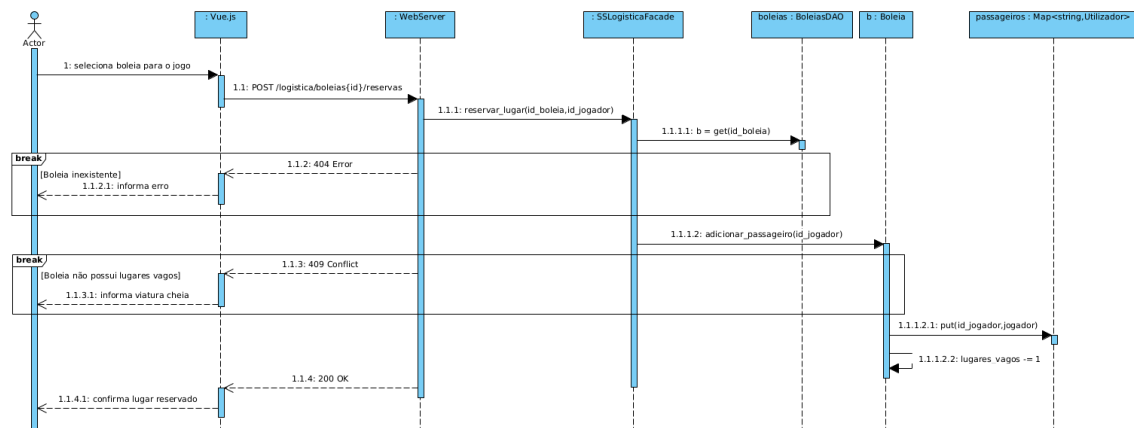


Figura 20: Diagrama de sequencia para reservar lugar em viatura

Este diagrama explicita a regra critica de decremento atomico dos lugares vagos, indispensavel para evitar reservas concorrentes sobre o ultimo lugar disponivel.

2.4.11. Diagrama de sequencia para Cancelar Reserva de Boleia

Prompt

Deves produzir um diagrama de sequencia para o caso de uso "Cancelar Reserva de Boleia". Deves seguir as restricoes seguintes:

- O ator principal e o Jogador.
- O fluxo deve passar por Vue.js, API Flask, SSLogisticaFacade, LogisticaDAO, Boleia e PostgreSQL.
- Deves mostrar a validacao do prazo limite e a remocao do passageiro.
- Inclui um ramo alternativo para limite temporal excedido.

O output correspondente encontra-se em diagramasseq/11_cancelar_reserva_boleia.mmd. Apos exportacao para imagem, recomenda-se a seguinte insercao:

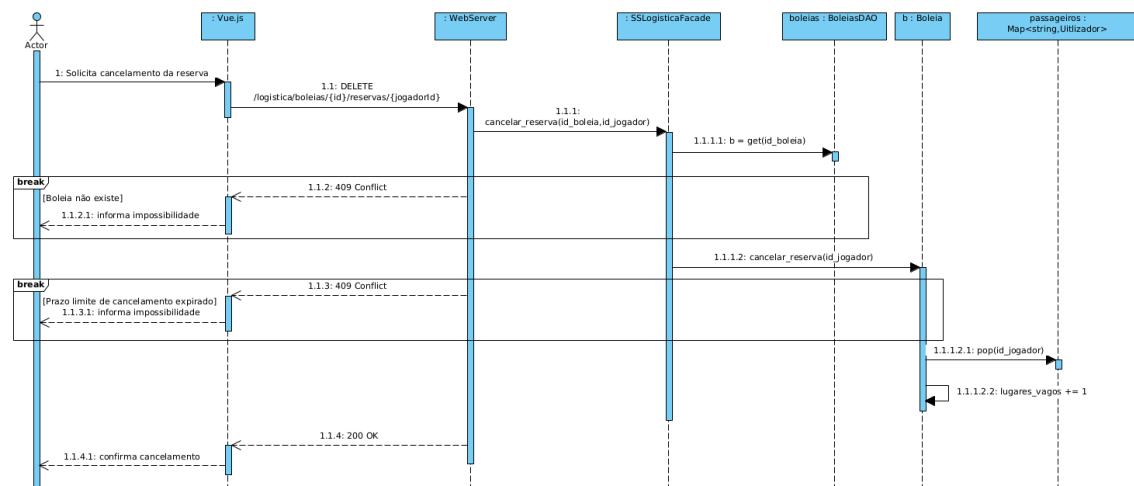


Figura 21: Diagrama de sequencia para cancelar reserva de boleia

O diagrama evidencia a regra temporal associada ao cancelamento da reserva, bem como a reposicao do lugar vago apos a remocao do passageiro.

Em suma, o conjunto de diagramas de sequencia agora produzido complementa os diagramas de classes anteriormente definidos, descrevendo o comportamento dinamico dos principais casos de uso do sistema e tornando mais clara a colaboracao entre os componentes que virao a ser implementados. A elaboracao dos diagramas de componentes sera efetuada posteriormente, com base nesta mesma estrutura arquitetural.

Para existir uma visão arquitetural geral, através de um diagrama de componentes, optamos por propor o seguinte prompt ao agente:

Prompt

Deves agora elaborar um diagrama de componentes, que possua 4 componentes principais.

- Um componente App Vue.js, que não deve possuir mais componentes dentro por se tratar da camada de apresentação.
- Um Componente WebServer, que representa o middleware que disponibiliza uma interface, à qual vamos denominar API Web
- Um componente OsBeiroesLN, que deve possuir 3 componentes no seu interior (SSGestaoFacade, SSEventosFacade e SSLogisticaFacade). Este componente deve disponibilizar uma interface IOSBeiroesLN, que deve ser consumida pelo anterior componente, WebServer
- Um Componente OsBeiroesDL, que deve possuir os componentes internos UtilizadoresDAO, EventosDAO, ComunicadosDAO e BoleiasDAO. Estes subcomponentes deve ser conectados, sempre através de portas, aos respectivos Facade que utilizam cada DAO.

Output

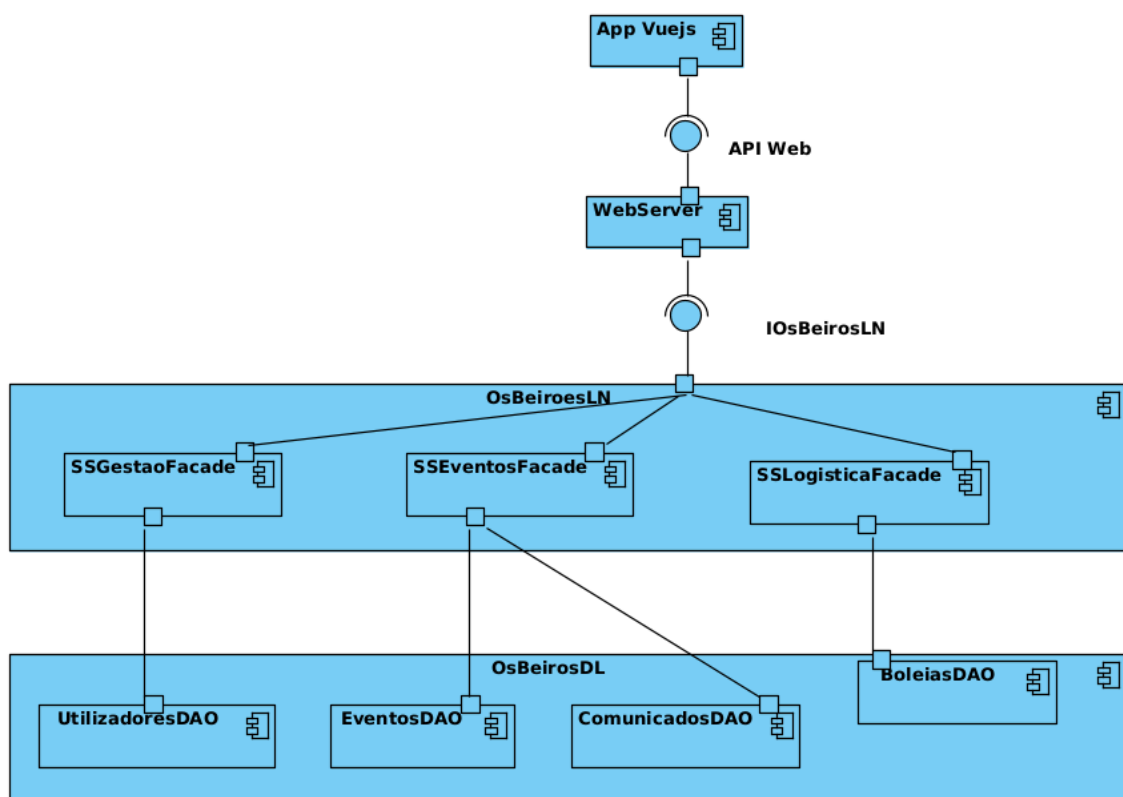


Figura 22: Diagrama de componentes para do sistema

Após uma análise, a equipa concluiu que o diagrama atende às expectativas, pelo que foi tomada a decisão de utilizar o diagrama.

2.5. Design de interfaces

Pretendendo gerar agora um design ou esboço das interfaces da aplicação, compusemos o seguinte prompt, fazendo uso da língua inglesa, para uma abordagem dos temas mais naturalmente.

You are building a mobile-first web application called "O Caderno do Quim" for Sport Clube "Os Beirões", a small, provincial Portuguese football club from the Vila de São Vicente da Beira. The system replaces:

Physical paper dossiers and scattered Excel files for player management

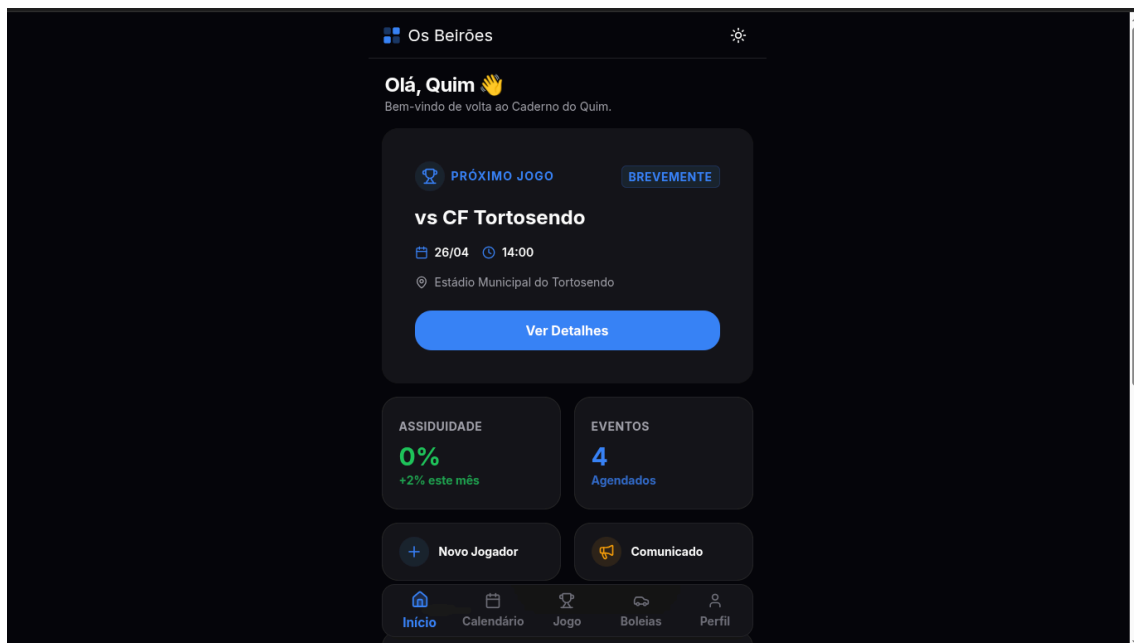
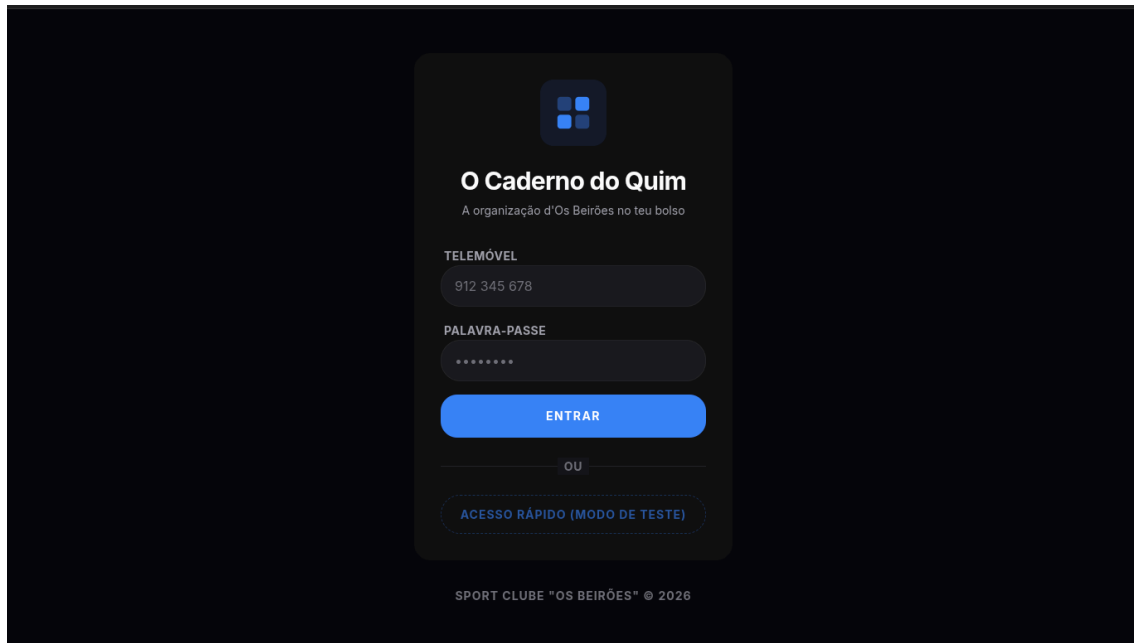
WhatsApp group chaos for convocations and communications

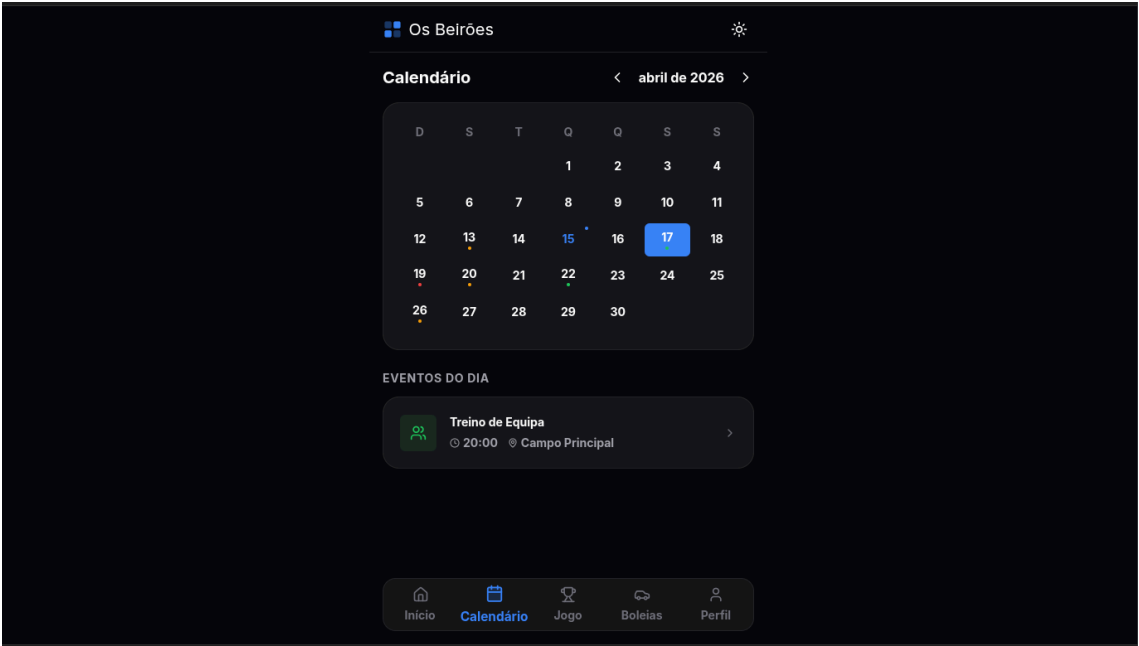
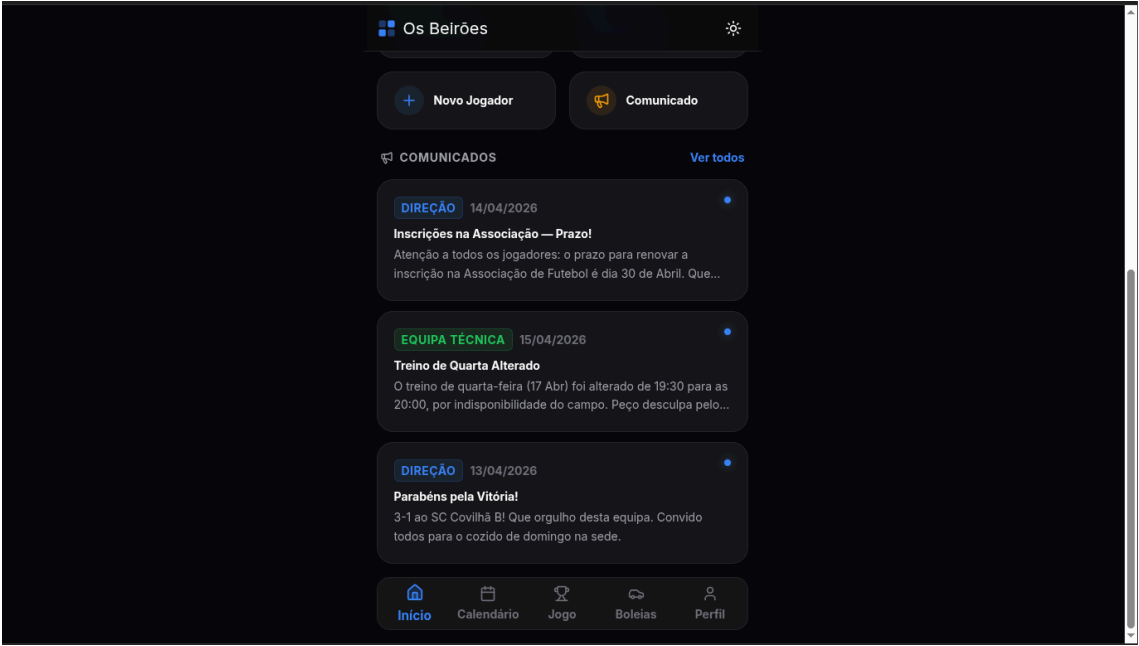
Last-minute word-of-mouth carpooling coordination for away games

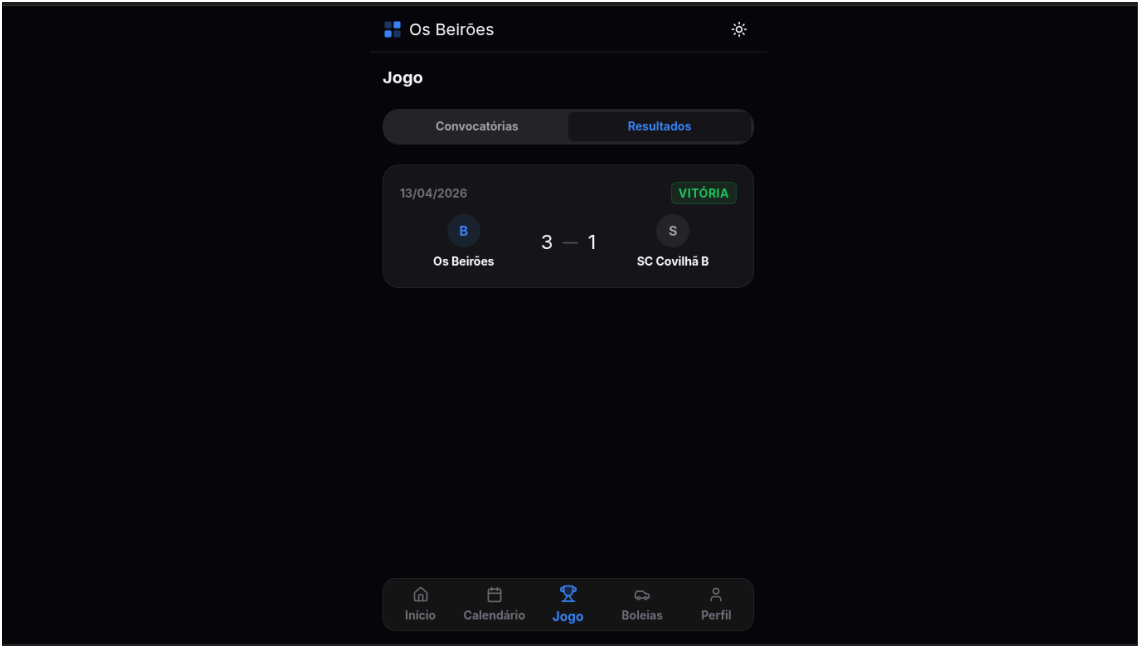
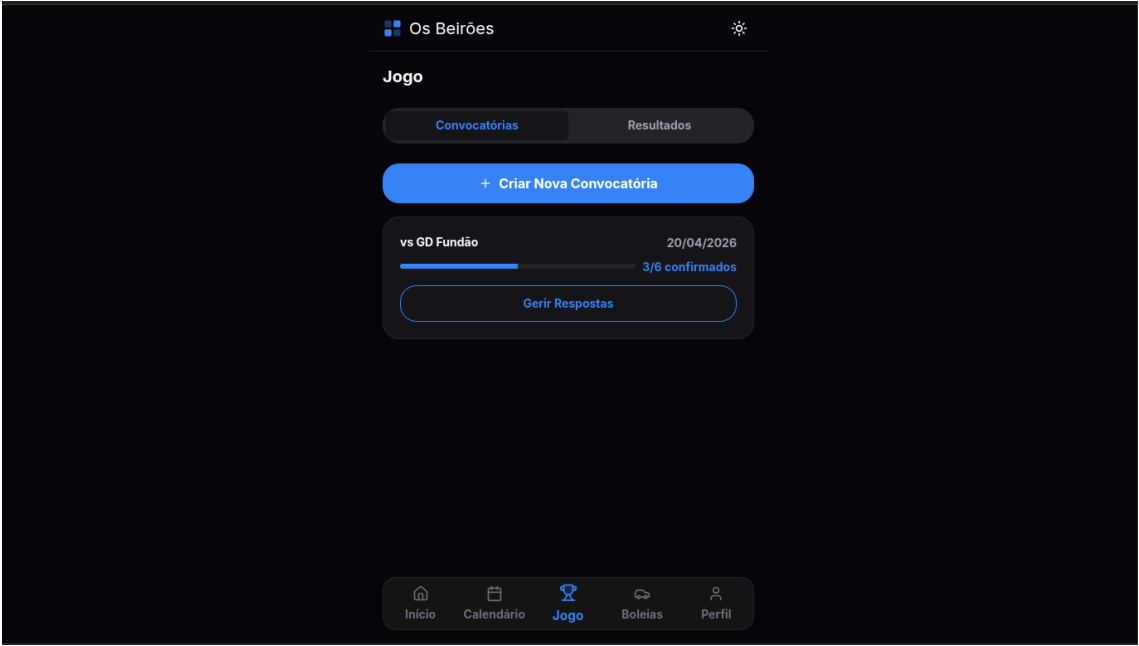
Zero attendance/training records

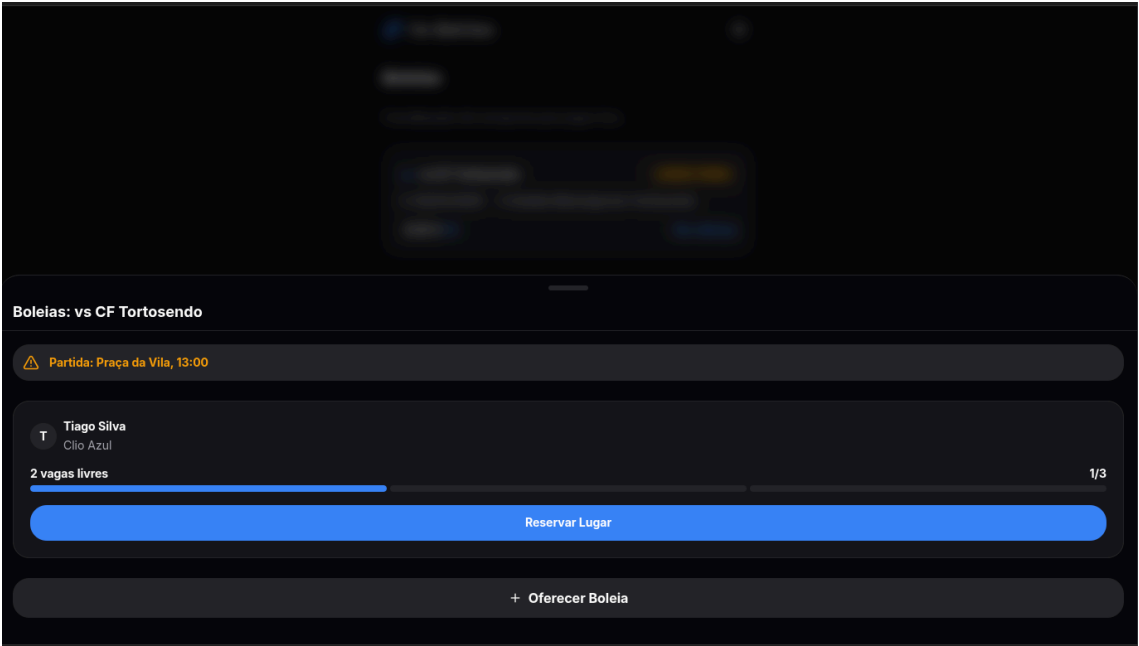
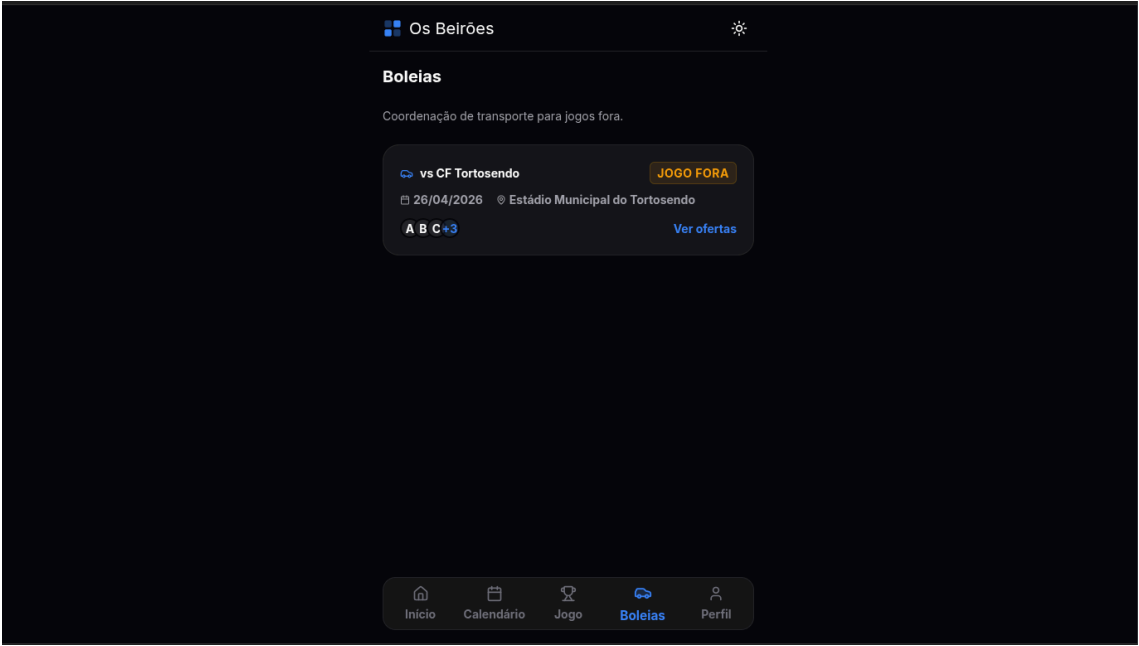
The application must feel like a digital "community notebook" – warm, approachable, and trusted by users who are NOT tech-savvy. The club's president, Sr. Joaquim "Quim" Barreira, and the coach, "Mister Zé", are the primary power users. Every design and UX decision must serve people who may have never used a professional app.

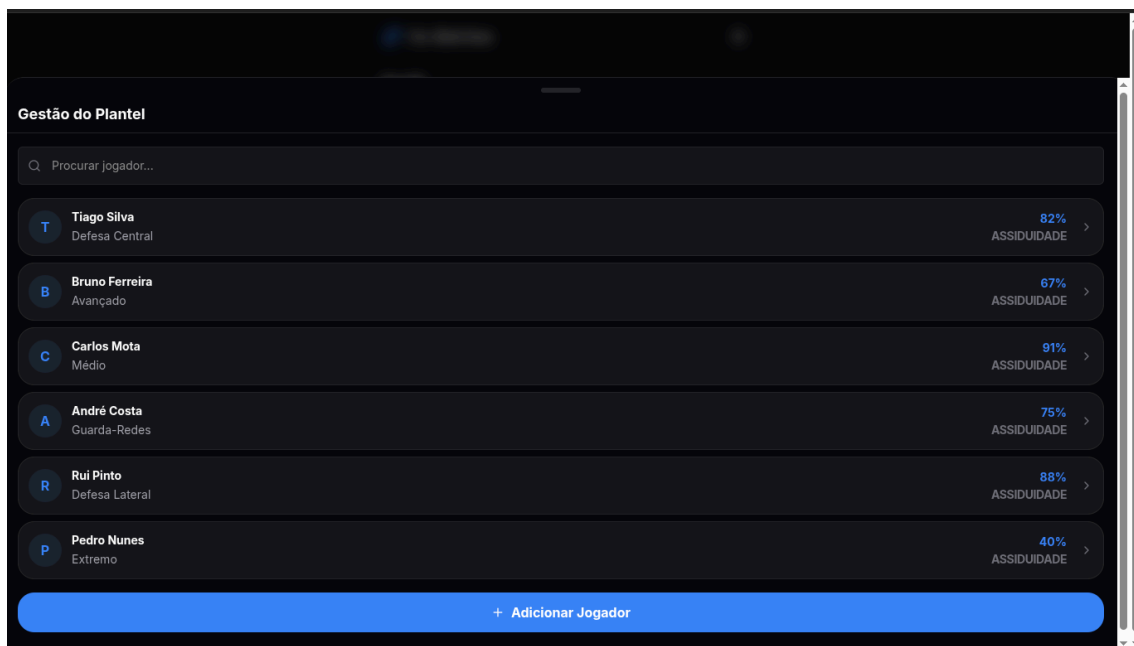
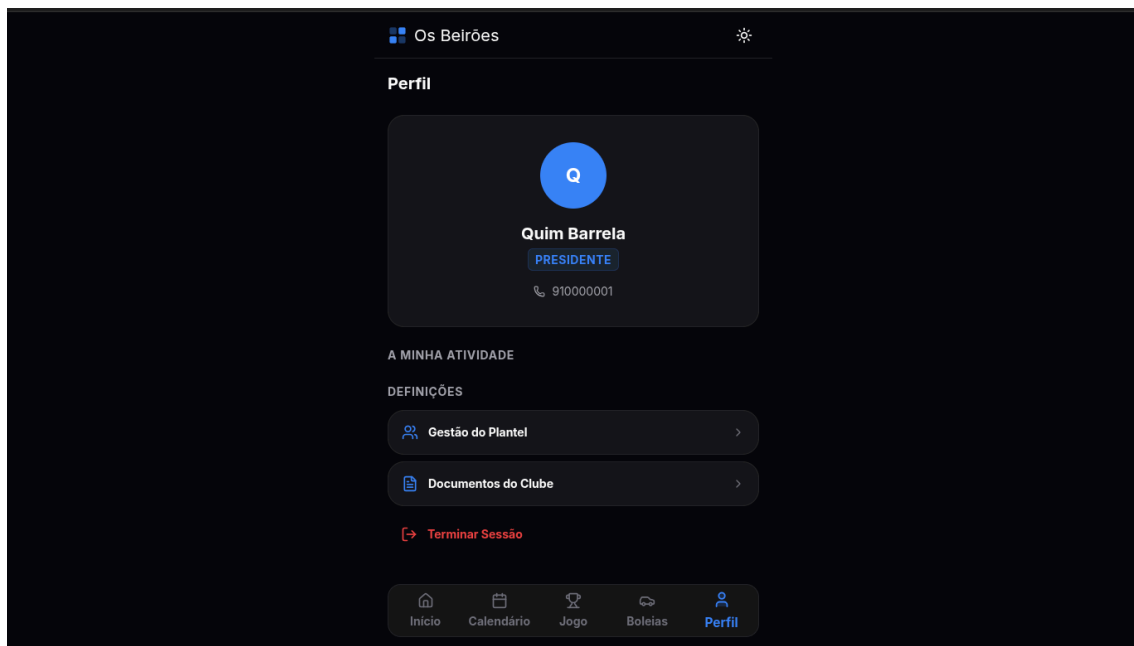
The final output must be a single self-contained HTML file with embedded CSS and JavaScript, ready to run in a browser without any server-side dependencies. All state management is in-memory. Use CDN libraries where needed.











2.6. Especificação de API e as decisões arquiteturais tomadas

Deves agora especificar a API web do sistema, que deve conter os endpoints explícitos nos diagramas de sequência deste relatório.

2.6.1. Subsistemas e Endpoints

2.6.1.1. Gestão de Utilizadores

Gere a autenticação e manutenção do plantel.

- **POST** /auth/login Autentica o utilizador e devolve um token de sessão.
- **POST** /jogadores/jogador Regista um novo jogador no sistema.

- **GET** /jogadores/procurar Permite procurar jogadores por nome ou identificador.
- **PUT** /jogadores/jogador/<id> Atualiza os dados de um jogador existente.
- **DELETE** /jogadores/jogador/<id> Inativa um jogador mantendo o histórico.

2.6.1.2. Calendário e Eventos

Responsável pelo agendamento e comunicação.

- **POST** /eventos Cria um novo evento (treino ou jogo).
- **PUT** /eventos/<id> Edita informações de um evento.
- **DELETE** /eventos/<id> Cancela um evento existente.
- **POST** /comunicados Publica um comunicado oficial.
- **GET** /comunicados Lista os comunicados disponíveis.

2.6.1.3. Operações Desportivas

Gere convocatórias, presenças e resultados.

- **POST** /eventos/jogos/<id>/convocatoria Define a convocatória para um jogo.
- **POST** /eventos/jogos/<id>/resposta Regista a resposta do jogador à convocatória.
- **POST** /eventos/treinamentos/<id>/presencas Regista as presenças num treino.
- **PUT** /eventos/jogos/<id>/resultado Regista o resultado final de um jogo.

2.6.1.4. Logística (Boleias)

Coordena transporte entre atletas.

- **POST** /boleias Cria uma nova oferta de boleia.
- **PATCH** /boleias/<id> Atualiza o número de lugares disponíveis.
- **POST** /boleias/<id>/reservar Reserva um lugar numa boleia.
- **DELETE** /boleias/<id>/reserva Cancela uma reserva existente.

2.6.2. Códigos de Estado HTTP

- **200 OK** Pedido executado com sucesso.
- **201 Created** Recurso criado com sucesso.
- **400 Bad Request** Pedido inválido ou violação de regras de negócio.
- **403 Forbidden** Utilizador sem permissões para executar a ação.

2.6.3. Notas de Implementação

- **Autenticação:** Todos os endpoints (exceto login) requerem um token JWT no cabeçalho: Authorization: Bearer <token>
- **Formato de Dados:** Comunicação baseada em JSON.
- **Persistência:** Operações de escrita persistem dados em base de dados relacional e podem acionar mecanismos de sincronização em cache.

2.6.4. Decisões arquiteturais

Com o intuito de justificar as decisões arquiteturais tomadas, a equipa elaborou o seguinte prompt

Prompt

Deves agora fundamentar e explicitar as decisões arquiteturais tomadas. Tudo deve ser devidamente justificado.

Output

A fundamentação da arquitetura do sistema “O Caderno do Quim” baseia-se na necessidade de equilibrar o rigor da engenharia de software com as restrições extremas do contexto (orçamento zero, hardware limitado e baixa literacia digital).

Abaixo, detalham-se as decisões tomadas, justificadas à luz da norma **ISO/IEC/IEEE 42010** e dos princípios de **Ian Sommerville**.

2.6.4.1. Modelo Arquitetural: 3-Layered Architecture (Abordagem “uminho”)

A decisão de estruturar o sistema em três camadas independentes (**Presentation, Logic, Data**) foi tomada para garantir a **manutenibilidade** e o **desacoplamento**.

Justificação: Ao contrário de uma arquitetura monolítica simples, esta divisão permite que a equipa altere a base de dados (ex: mudar de PostgreSQL para um serviço Cloud no futuro) ou a interface (Vue.js) sem nunca tocar na lógica de negócio central (os subsistemas). Isto protege o investimento em código, algo vital num projeto de orçamento inexistente.

2.6.4.2. Padrão de Fachada (Facade Pattern) e Subsistemas

O sistema está dividido em quatro subsistemas atômicos: **Gestão, Eventos, Operações e Logística**. Cada um expõe uma **Fachada (Facade)**.

Justificação:

Encapsulamento: Segundo Sommerville, a complexidade deve ser escondida. O BeiroesLNFacade (Fachada Global) atua como um ponto único de entrada, impedindo que a camada Web (Flask) conheça os detalhes internos de como uma boleia é processada. **Atomicidade:** Garante que cada subsistema lide apenas com a sua responsabilidade, facilitando o desenvolvimento paralelo e os testes.

2.6.4.3. Camada de Dados: Padrão DAO (Data Access Object)

Inspirado no modelo FastDL, cada subsistema utiliza **DAOs** para interagir com a persistência.

Justificação:

Abstração de Persistência: O uso de DAOs (ex: EventoDAO) permite que a lógica de negócio manipule objetos em memória (via cache) de forma transparente. Para o Raspberry Pi, isto é crítico: minimiza-se o acesso ao disco (cartão SD), aumentando a longevidade do hardware. **Agregação de Objetos:** Decidiu-se carregar objetos completos (ex: uma Convocatória que contém instâncias de Jogador) em vez de apenas IDs. Isto reduz o número de “joins” complexos no PostgreSQL, tornando a aplicação mais rápida em hardware limitado.

2.6.4.4. Escolha Tecnológica: Python (Flask) e Vue.js (CSR)

Flask: Escolhido pela sua natureza de **micro-framework**. É leve o suficiente para correr num Raspberry Pi sem sobrecarregar o CPU, permitindo um desenvolvimento muito mais célere que o Java num contexto de prototipagem rápida.

Vue.js com CSR: Ao processar a interface no cliente (telemóvel do jogador), o servidor limita-se a enviar dados JSON.

Justificação: Isto resolve o problema da “névoa de incerteza” do calendário. A interface reage instantaneamente, dando ao utilizador a sensação de uma app nativa, o que é crucial para a aceitação por parte de utilizadores menos letrados digitalmente.

2.6.4.5. Infraestrutura: Raspberry Pi + Cloudflare Tunnel

Raspberry Pi: A decisão fundamenta-se no custo único de aquisição e consumo energético quase nulo.

Cloudflare Tunnel: Resolve o problema de conectividade na Beira Interior (onde IPs fixos são raros).

Justificação: Permite expor a API de forma segura (HTTPS) sem necessidade de abrir portas no router da sede do clube, protegendo o sistema de ataques externos triviais.

2.6.4.6. Política de Backup e Continuidade

A política de backup diário (Local + Nuvem) é a resposta à **criticidade dos dados** (ex: contactos de emergência).

Justificação: Sommerville define a **Disponibilidade** como um atributo de qualidade essencial. Como o Raspberry Pi é um ponto único de falha física, o processo automático garante que o “Caderno do Quim” possa ser restaurado em qualquer outro computador em menos de 2 horas.

2.6.4.7. Conclusão das Decisões

Esta arquitetura transforma um cenário de “carolice” numa solução profissional e resiliente. O uso de padrões formais (Fachadas, DAOs, Camadas) garante que o sistema não seja apenas uma “ferramenta técnica”, mas um ativo duradouro para o Sport Clube “Os Beirões”, capaz de sobreviver à baixa literacia e à falta de fundos.

Após analisar a resposta do agente, a equipa entendeu que as decisões tomadas até então foram explicitadas e fundamentadas segundo a conceção da arquitetura que a equipa tinha idealizado. Por isso, optamos por adotar o output do agente.

3. Implementação e Desenvolvimento Assistido por LLM

3.1. Configuração do ambiente de desenvolvimento

Com o intuito de garantir a reprodutibilidade do projeto e o isolamento das dependências tecnológicas, procedeu-se à configuração do ambiente de trabalho. Para este efeito, utilizou-se a tecnologia de conten-torização para assegurar que o sistema corre de forma idêntica no computador de desenvolvimento e no Raspberry Pi de destino.

Prompt

Atuando como um engenheiro de software, deves configurar o ambiente de desenvolvimento para o projeto "O Caderno do Quim".
Cria um ficheiro docker-compose.yml que inclua um serviço para o backend em Flask (Python 3.11), um serviço para a base de dados PostgreSQL e um volume para persistência.
Deves também gerar o Dockerfile para o backend, garantindo a instalação das dependências a partir de um requirements.txt.
Justifica a escolha destas tecnologias com base na restrição de portabilidade e facilidade de deploy num Raspberry Pi.

Analisando o output, a equipa verificou que a utilização de Docker permite mitigar o risco de conflitos de bibliotecas no sistema operativo do Raspberry Pi, garantindo que o ambiente de execução é controlado e fácil de restaurar em caso de falha do hardware.

3.2. Implementação incremental e modular do Subsistema de Eventos

Prosseguindo para a fase de codificação, adotou-se uma estratégia de desenvolvimento modular, focando inicialmente no **Subsistema de Eventos**. Esta decisão deve-se à criticidade do calendário para resolver a “névoa de incerteza” mencionada pelos stakeholders.

Prompt

Baseando-te no diagrama de classes e de sequência do Subsistema de Eventos, gera a implementação em Python (Flask) seguindo o padrão Facade e DAO.
Deves implementar:

1. A classe de domínio Evento (com subclasses Treino e Jogo).
2. A interface IEventosDAO e a sua implementação EventoDAO para PostgreSQL.
3. A SSEventosFacade que encapsula a lógica de agendamento e validação de conflitos de horário.

O código deve ser limpo e incluir docstrings explicativas.

Após a geração, a equipa validou que a lógica de negócio está devidamente isolada da persistência através do DAO, respeitando as decisões arquiteturais de desacoplamento tomadas anteriormente.

3.3. Geração de código assistida e revisão automática

Com o objetivo de garantir a qualidade do código e a ausência de vulnerabilidades, submeteu-se a implementação a um processo de revisão assistida por LLM. Este passo visa detetar problemas que ferramentas de análise estática convencionais poderiam ignorar.

Prompt

Atua como um revisor de código (code reviewer). Analisa o código do Subsistema de Eventos gerado anteriormente.

Deves identificar:

- Code Smells (ex: funções demasiado longas ou acoplamento excessivo).
- Possíveis falhas de segurança (ex: SQL Injection na camada DAO).
- Eficiência táctica (ex: excesso de acessos ao disco no Raspberry Pi).

Apresenta as sugestões de melhoria num formato de lista e justifica cada alteração com base em Ian Sommerville.

A equipa notou que o agente sugeriu a implementação de uma cache em memória no DAO para as consultas frequentes ao calendário, minimizando o desgaste do cartão SD do Raspberry Pi, o que vai ao encontro da política de resiliência definida.

3.4. Refatoração e deteção de smells

Para finalizar o ciclo de desenvolvimento do módulo, procedeu-se à refatoração do código baseando-se nas críticas da fase anterior. O foco incidiu na simplificação das interfaces para garantir que a API Web é atómica e eficiente.

Prompt

Com base na revisão anterior, procede à refatoração (refactoring) do SSEventosFacade.

Deves simplificar o método de validação de conflitos de horário utilizando as sugestões de otimização de memória.

Garante que a API Web (Flask) consome esta fachada de forma transparente e que os erros de negócio (ex: data no passado) são devolvidos com os códigos HTTP adequados.

Analisando a versão final, concluiu-se que o código refatorado apresenta uma maior manutenibilidade e robustez operacional, estando pronto para a integração com o Front-end em Vue.js.

4. Conclusões e Trabalho Futuro

Referências

Lista de Siglas e Acrónimos

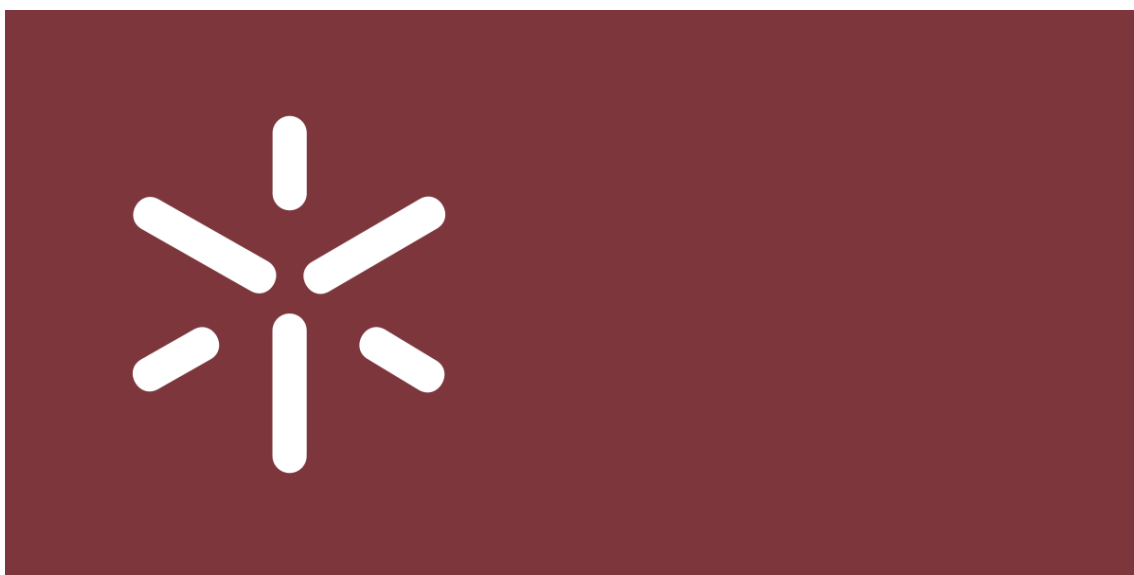
BD Base de Dados

DW Data Warehouse

OLTP On-Line Analytical Processing

Anexos

Anexo 1: Logo da Universidade do Minho



Bibliografia

[1] I. Sommerville, *Software Engineering*, 9.^a ed. Addison-Wesley, 2010, p. 792.